EXPANDED AND VERIFIED EDITION

Agent Harness Anatomy

A field guide to thirteen AI agent harnesses —
how they loop, call tools, decide what is allowed, and fail

Based on the series by D. Marques (github.com/dpsm/agent-harness-anatomy, Apache-2.0)
Expanded, source-verified, and annotated with a security lens
Prepared for Patrick Campbell · October 12, 2026

Facts are pinned to the commits listed in each chapter and were re-checked against upstream source on October 12, 2026.

Contents

About This Edition **5**
How to Read the Chapters 5

The Landing Page, Verified **7**
What Was Confirmed 7
What Needs Correcting 8

What an Agent Harness Is **9**
The Harness, Defined 9
The Method: Pin Everything 9
The Ten Dimensions 9
Themes to Watch For 10

1. pi — The Minimalist Terminal Harness **11**
In One Paragraph 11
Mental Model 11
A Turn, Step by Step 12
Subsystems Worth Knowing 12
Design Tradeoffs 13
Security Lens 13
Take-away 14

2. aider — The Pair Programmer That Parses Prose **15**
In One Paragraph 15
Mental Model 15
A Turn, Step by Step 16
Subsystems Worth Knowing 16
Design Tradeoffs 17
Security Lens 17
Take-away 18

3. Cline — The Approval-Gated IDE Resident **19**
In One Paragraph 19
Mental Model 19
A Turn, Step by Step 20
Subsystems Worth Knowing 20
Design Tradeoffs 21
Security Lens 21
Take-away 22

4. Goose — The MCP-Native Agent That Asks a Judge **23**
In One Paragraph 23
Mental Model 23
A Turn, Step by Step 24
Subsystems Worth Knowing 24
Design Tradeoffs 25
Security Lens 25
Take-away 26

5. Codex CLI — The Sandbox-First Agent That Speaks One Wire Protocol **27**
In One Paragraph 27
Mental Model 28
A Turn, Step by Step 28
The Sandbox Stack 28
Subsystems Worth Knowing 29
Design Tradeoffs 29
Security Lens 30

Take-away 30

6. OpenHands — The Event-Sourced Agent SDK 31

In One Paragraph 31
Mental Model 32
A Turn, Step by Step 32
Subsystems Worth Knowing 32
Design Tradeoffs 33
Security Lens 33
Take-away 34

7. OpenCode — The Dual-Runtime Coding Agent 35

In One Paragraph 35
Mental Model 36
A Turn, Step by Step 36
Subsystems Worth Knowing 36
Design Tradeoffs 37
Security Lens 37
Take-away 37

8. Hermes Agent — The Self-Improving Agent 38

In One Paragraph 38
Mental Model 39
A Turn, Step by Step 39
The Learning Loop 39
Subsystems Worth Knowing 40
Design Tradeoffs 40
Security Lens 40
Take-away 41

9. OpenClaw — The Channel-First Gateway 42

In One Paragraph 42
Mental Model 42
A Message, Step by Step 43
The Four-Layer Loop 43
Subsystems Worth Knowing 44
Design Tradeoffs 44
Security Lens 44
Take-away 45

10. Gemini CLI — The Routed, Policy-Gated Terminal Agent 46

In One Paragraph 46
Mental Model 47
A Turn, Step by Step 47
Subsystems Worth Knowing 48
Design Tradeoffs 48
Security Lens 49
Take-away 49

11. SWE-agent — The Research Original 50

In One Paragraph 50
Mental Model 51
A Step, End to End 51
Subsystems Worth Knowing 51
Design Tradeoffs 52
Security Lens 52
Take-away 52

12. Qwen Code — The Fork That Outgrew Its Parent 53

In One Paragraph 53
Mental Model 53
A Turn, Step by Step 54
Subsystems Worth Knowing 54

Design Tradeoffs 55
Security Lens 55
Take-away 56

13. Darwin Gödel Machine — The Agent That Rewrites Itself 57

In One Paragraph 57
Mental Model 58
The Outer Loop 58
One Child, End to End 58
The Inner Agent 59
Honest Limits 59
Security Lens 59
Take-away 60

14. The Thirteen, Side by Side 61

Who Stops the Loop? 61
Ask, Contain, or Undo? 61
How Much of the Tool Catalog Does the Model See? 62
Where MCP Plugs In 62
A Second Model in the Safety Path 63
Patterns Worth Stealing 63

15. Found a Mistake? 64

Why Claims Drift 64
How to File a Good Correction 64
Corrections From This Review 64

Appendix A — Verification Evidence 67

Appendix B — Glossary 69

Appendix C — Sources 70

About This Edition

This guide takes the landing page of **Agent Harness Anatomy** (dpsm.github.io/agent-harness-anatomy) and does three things to it:

1. **Checks it.** Every summary on the landing page was compared against the series' own articles and then against the actual source code of all thirteen harnesses, downloaded at the exact commits the series pins.
2. **Expands it.** Each one-paragraph summary on the landing page becomes a full chapter: the mental model, a turn traced step by step, the subsystems that matter, the design tradeoffs, and what to take away if you build or assess harnesses yourself.
3. **Adds a security lens.** Each harness chapter ends with notes on where trust boundaries sit, where defaults fail open, and what an assessor should probe first. These notes stay at the design level and are drawn from the public source.

> **Source and license.** The original series is by D. Marques (github.com/dpsm/agent-harness-anatomy), published under the Apache License 2.0. This edition is a derived, expanded work. Facts about each harness come from the series' articles and from the pinned upstream code; analysis, the security notes, and the corrections are new. Repository snapshot used: commit `6079e18` (October 11, 2026).
>
> **Verification date.** October 12, 2026. All thirteen pinned commits resolved on GitHub. Where this guide disagrees with the original, the chapter says so in a "Verified" box and the full evidence is in Appendix A.

How to Read the Chapters

Every harness chapter follows the same shape so you can compare them side by side:

- **Pinned version** — the release and commit the facts apply to. Harnesses change quickly; a claim is only as good as its pin.
- **Verified** — what was checked against code, and any correction.
- **In one paragraph** — the landing-page idea, restated plainly.
- **Mental model** — how to think about the harness before reading any code.
- **A turn, step by step** — one user request traced from input to output.
- **Subsystems worth knowing** — tools, approvals, memory, providers, extensibility.
- **Design tradeoffs** — what the harness gains and what it pays.
- **Security lens** — trust boundaries, fail-open seams, and first things to probe.
- **Take-away** — the one idea worth stealing.

Chapter 14 compares all thirteen in tables. Chapter 15 expands the original "Found a mistake?" note and lists the corrections from this review in a form that could be filed upstream.

The Landing Page, Verified

The short answer: **the landing page is accurate on almost every specific claim.** The numbers it leads with were all reproduced from the pinned code. Eight items need correction or tightening; none of them changes the overall picture of a harness.

What Was Confirmed

#	Harness	Landing-page claim checked	Result
1	pi	No iteration cap; no built-in approvals; prompt sections diffed per turn	Confirmed (0 cap symbols; `diffSystemPromptSections`, tool deltas)
2	aider	313-entry model table; PageRank repo map; git as undo	Confirmed (313 entries; `nx.pagerank`; auto-commit + `/undo`)
3	Cline	224 RPCs; three-layer approvals; stash checkpoints	Confirmed (22 services, 224 RPCs; `git stash create` under `refs/cline/checkpoints`)
4	Goose	Every tool is MCP; LLM judge for SmartApprove; two loops	Confirmed (4 extension types, no SSE; "UNTRUSTED TOOL REQUEST DATA" judge; `GOOSE_STATE_MACHINE`)
5	Codex CLI	One JSON-RPC app-server; no max_turns; bwrap + landlock + seccomp; hooks outrank user	Confirmed (exec runs an in-process app-server; precedence comment "1. Hooks")
6	OpenHands	Typed event tree; critic veto; per-completion routing	Confirmed (5 event types; `RouterLLM`; refinement at finish)
7	OpenCode	Tools rebuilt per request; LSP feedback inside the tool call	Confirmed (`usePatch` swap; 20-error cap per file)
8	Hermes Agent	Background review; curator on a slower timescale	Confirmed (7-day curator interval; archive, never delete)
9	OpenClaw	Seven-stage `runChannelTurn`; SQLite ingress queue with dead letters	Confirmed (stage events in that exact order)
10	Gemini CLI	Per-turn router; TOML policy tiers; 11 hooks; no tool search; 26 built-ins	Confirmed, with one correction below
11	SWE-agent	ACI defined as data; per-step trajectory; reviewer retry loops	Confirmed (YAML bundles; `.traj` saved every step)
12	Qwen Code	Forked before policy engine and router; 13 presets; 22 hooks	Confirmed (README names v0.8.2; 13 presets; 22 hook events)
13	Darwin Gödel Machine	Archive, not hill-climbing; diagnoser sees test patch; v3 postdates pin	Confirmed (`{test_patch}` in prompt; arXiv v3 dated March 12, 2026)

What Needs Correcting

#	Where	What the site says	What the code shows
C1	Landing page, Gemini CLI	Memory is "mined nightly"	Extraction runs when the CLI starts, at most once every 30 minutes, on sessions idle for 3+ hours with 10+ user messages. Not nightly.
C2	Comparison matrix, Codex	14 `RolloutItem` variants	12 variants. The article body already says 12; the matrix cell is wrong.
C3	Hermes article	`max_iterations` defaults to 500	500 is the CLI's config default. The agent class defaults to unlimited, and the gateway falls back to unlimited when no limit is set.
C4	SWE-agent article	"The ancestor OpenHands descends from"	Not supported. They were separate projects in spring 2024; the one code link found runs the other way.
C5	Landing page, OpenCode	"Runs its session loop on two runtimes"	One session loop, two LLM transports underneath. The article itself describes it correctly.
C6	Landing page, OpenHands	"Microagents are dead"	Still loaded from `~/.openhands/microagents/` as legacy and converted into Skills. Deprecated, not removed.
C7	Landing page, DGM	"Sakana AI's"	The paper is a University of British Columbia, Vector Institute, and Sakana AI collaboration.
C8	Landing page, Gemini CLI	"All 26 built-ins ride every request"	26 is the most that can be registered at once; the to-do, plan-mode, and tracker tools depend on settings. "Every active tool rides every request" is exact.

Two further notes are matters of wording, not fact: the landing page says Hermes' curator "promotes" skills (the code pins, archives, and consolidates), and the Hermes article's "7 days idle" means "7 days since the last curator run, while the agent has been idle at least 2 hours."

What an Agent Harness Is

The landing page opens with one line of purpose: source-grounded architecture teardowns of AI agent harnesses, with every behavioral claim pinned to verifiable source code. This chapter unpacks what that means and why the method matters.

The Harness, Defined

A language model on its own produces text. An **agent harness** is everything around the model that turns that text into action: the loop that calls the model again and again, the tools it can call, the code that decides whether a tool call is allowed, the memory that survives between turns, and the interfaces a person uses to drive it.

Two products running the same model can behave very differently because their harnesses differ. One stops after 100 turns; another never stops on its own. One asks before every shell command; another runs commands inside a kernel sandbox and never asks; a third asks nobody and relies on git to undo mistakes. Those choices live in the harness, not the model. That is why the series studies harnesses rather than models.

The Method: Pin Everything

Harness code changes weekly. A blog post that says "Tool X asks before running commands" is only true for some range of versions, and readers rarely know which. The series handles this with three rules:

1. **Pin a release tag and commit.** Every chapter states the exact commit its facts apply to.
2. **Link every behavioral claim to a line of code.** Not to documentation, not to a README, but to a permalink inside the pinned tree.
3. **Grade each claim.** The series uses three grades: VERIFIED (read from source), DOCS (stated in documentation only), and INFERRED (reasoned from source but not directly shown). The grade travels with the claim.

The payoff shows up repeatedly in the chapters. Cline's documentation describes a checkpoint system that the code no longer uses. Goose's design docs describe an `Extension` trait that does not exist. SWE-agent's docs describe a file viewer that the default configuration does not load. In each case, code wins, and the disagreement is itself a finding.

The Ten Dimensions

Each teardown fills in the same ten-row comparison, which makes the harnesses comparable even when their architectures look nothing alike:

#	Dimension	The question it answers
1	Agent loop	How does the harness decide to call the model again, and what stops it?

#	Dimension	The question it answers
2	Tool system	How are tools declared, validated, and executed?
3	Model providers	Which models can it talk to, and how?
4	Prompt construction	What exactly does the model see, in what order?
5	Memory and session	What persists between turns and between sessions?
6	Reasoning and planning	Are there planners, sub-agents, judges, or critics?
7	Extensibility	Where can someone add behavior without forking?
8	Interfaces	Terminal, IDE, web, messaging, SDK?
9	Failure handling	Retries, truncation, loops, crashes?
10	Security model	Approvals, sandboxes, undo, trust?

Themes to Watch For

Reading all thirteen teardowns, a few questions come up again and again. Keep them in mind as you read:

- **Who stops the loop?** pi, Codex, OpenCode, OpenClaw, SWE-agent, and DGM have no hard turn cap in the core loop. Goose caps at 1,000 turns, OpenHands at 500 per run, Gemini CLI and Qwen Code at 100 per request. Cline caps repetition instead of turns.
- **Ask, contain, or undo?** Safety comes in three flavors. Cline and Goose *ask* (approvals, judges). Codex *contains* (an operating-system sandbox). aider and Cline *undo* (git commits and stash snapshots). Most harnesses pick one or two; none does all three well.
- **How much of the tool catalog does the model see?** Goose, Hermes, OpenClaw, Qwen Code, Codex, and pi can hide tools until the model searches for them. Gemini CLI sends every active tool on every request.
- **Where does MCP plug in?** Goose makes every tool an MCP server. Cline and Gemini CLI mint MCP tools as native tools. pi and Codex wrap them. aider has no MCP at all.
- **Are skills prompts or tools?** Most harnesses inject skills as prompt text; Cline and OpenHands also expose them as callable tools; Hermes lets the agent write its own.
- **Who judges whom?** Several harnesses now put a second model call in the safety path: Goose's SmartApprove judge, Codex's Guardian reviewer, Gemini's Conseca, Qwen's AUTO classifier, OpenHands' security analyzer and critic. Each is a new trust boundary, and each has to decide whether failure means "allow" or "deny."

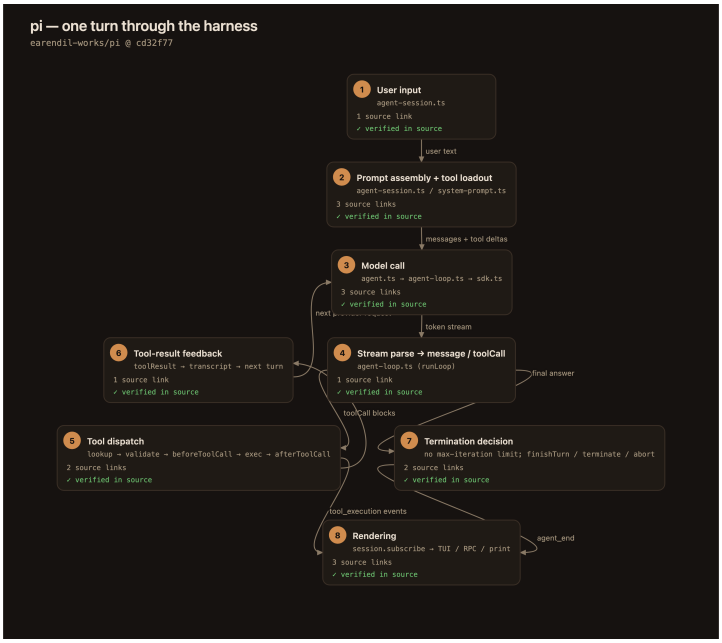
1. pi — The Minimalist Terminal Harness

Repo: earendil-works/pi · Pinned: v1.0.2 → cd32f77 (Oct 4, 2026) · TypeScript, Node 22.19+

> **Verified.** Eight built-in tools with four on by default (`read, bash, edit, write`); 42 provider catalogs; zero iteration-cap symbols in the loop or session packages; structured system-prompt sections diffed per turn with tool additions and removals sent as deltas; `--approve` controls project trust, not per-command approval. No corrections.

In One Paragraph

pi is two packages with one clean seam: a loop library that knows nothing about terminals, and a terminal coding app that configures one agent per session. The core supplies mechanism; extensions supply policy. There is no iteration cap and no built-in approval prompt, and the system prompt is sent as structured sections that are diffed each turn so only changes go over the wire.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

Think of pi as a REPL for a model. The loop package (`pi-agent-core`) only knows a protocol: look up a tool, validate its arguments, call a `beforeToolCall` hook, run the tool, call `afterToolCall`, append the result, and ask the model again. It does not know what any tool does. Everything specific to coding — the system prompt sections, the four default tools, the TUI — lives above the seam in the coding-agent package.

The governing idea is that anything opinionated belongs outside the core. Approvals, plan mode, sub-agents, and iteration caps are not missing by accident; the repo ships each of them as an *example extension*. You get a small, stable core and a large extension surface, and you pay by assembling your own policy layer.

The loop is event-sourced. Every state change emits a typed event (agent start and end, turn start and end, message updates, tool execution updates). The TUI, the RPC server, and print mode are all just subscribers to that stream. There is no privileged renderer.

A Turn, Step by Step

Suppose the user types "help me refactor foo.ts."

1. **Input.** The session gives extension slash commands the first chance to handle the text, then extension input handlers, then expands skill commands and prompt templates. If a run is already streaming, the message must be queued as either a *steer* (interrupt) or a *follow-up* (wait); calling without saying which throws an error.
2. **Prompt assembly.** Extensions may edit the system-prompt options. pi then compares the structured prompt — sections such as `preamble`, `tools`, `rules`, `docs`, `skills`, `cwd` — against what the transcript already contains and sends only the changed sections. A `null` removes a section. Each non-preamble section is wrapped in an XML tag of the same name so later updates can be matched.
3. **Model call.** The app's stream function delegates to the model runtime, which streams tokens from whichever of the ~42 providers is selected and warms the prompt cache.
4. **Parsing.** Tool calls are pulled from assistant content blocks. If the model stopped because it hit the output-token limit, pi does *not* execute the tool calls — each gets an error result, because the arguments may be truncated.
5. **Dispatch.** Tools run in parallel by default. Each call goes through lookup, argument preparation, TypeBox schema validation, `beforeToolCall`, execution, and `afterToolCall`. In the coding app, `beforeToolCall` only forwards to extension `tool_call` handlers.
6. **Feedback.** Results are appended as tool-result messages and the inner loop asks again.
7. **Termination.** The turn ends when no tool calls, steering, or follow-ups remain; when a `finishTurn` hook says stop; when every result in a batch sets `terminate`; or on error or Ctrl+C. Afterward the session retries retryable errors, checks compaction, and gives extensions a final `agent_before_settle` hook that can force another turn.

Subsystems Worth Knowing

- **Tools.** Eight built-ins. `powershell`, `grep`, `find`, and `ls` ship disabled. Each tool contributes a one-line snippet and guideline bullets to the prompt. Tools can call other tools through `ctx.executeTool()`, and nested calls re-enter the same hook pipeline.
- **Tool exposure.** Every tool has an exposure level: `direct`, `model-only`, `codemode`, `deferred`, or `hidden`. MCP tools default to `codemode` — callable from scripts but not declared to the model — and `deferred` tools surface only through `tool_search`.

- **Providers.** About 42 providers behind a single stream-function signature. API keys resolve per request, which supports expiring credentials.
- **Sessions.** JSONL files under `~/.pi/agent/sessions/`. Compaction includes branch summarization and runs before and after each prompt.
- **Extensions.** TypeScript modules dropped in `~/.pi/agent/extensions/` or a project's `.pi/extensions/`, with hooks at every seam: `input`, `before_agent_start`, `tool_call`, `tool_result`, `context`, `agent_before_settle`. Returning `{ block: true }` from a `tool_call` handler stops the call.
- **Skills.** Markdown files with `name` and `description` frontmatter. Listed in the prompt's `skills` section (only when `read` or `bash` is active, since the model needs one of them to open the file) and inlined when invoked as `/skill:name`.
- **Remote protocol.** A transport-neutral CBOR RPC protocol (version 8) carried over Unix sockets locally and over WebSocket through a relay.

Design Tradeoffs

- **No cap, by design.** A runaway model burns tokens until the user aborts. Anyone who wants a cap can add one in a `finishTurn` hook without forking. pi bets that a policy-free core stays small while the ecosystem sorts out policy.
- **Diffed prompts versus simple prompts.** Sending only changed sections keeps long sessions cheap, but the system message becomes a small versioned protocol that every provider adapter must replay correctly. Most harnesses resend everything and pay the tokens.
- **Event schema as contract.** Any interface can render a run without the loop knowing it exists, but adding an event type is now a versioning decision.
- **"Minimal" core, broader default install.** The coding app loads MCP, `codemode`, and `tool_search` as built-in extensions, so the core is minimal while a fresh install is somewhat larger.

Security Lens

- **There is no gate unless you install one.** Out of the box, the model can run `bash` with no per-command prompt. The trust decision is project-level (`--approve` trusts project-local files). An assessment should start by listing which extensions are loaded, because they *are* the security model.
- **Extensions run in-process.** A malicious or buggy extension has the agent's full privileges and can veto or rewrite anything. Treat extension directories as code you execute.
- **Project-local files are an input channel.** Skills, prompt templates, and extensions can come from the project directory. That is a prompt-injection and code-execution path when opening untrusted repositories; it is exactly what the project-trust flag exists to control.
- **Truncation guard is a good pattern.** Refusing to execute tool calls cut off by the output limit closes a real class of half-written-command bugs.

Take-away

Mechanism in the core, policy at the edges. If you are building a harness and want it to stay hackable, steal pi's hook pipeline and its exposure levels. If your users need guardrails on day one, pi gives you the hook, not the feature — budget for writing the gate.

2. aider — The Pair Programmer That Parses Prose

Repo: Aider-AI/aider · Pinned: v0.86.2 → 253f036 (Feb 11, 2026) · Python · Apache-2.0

> **Verified.** `model-settings.yml` has exactly 313 entries; `max_reflections = 3`; retry gives up past 60 seconds; 42 slash commands; the repo map calls `networkx.pagerank`; the edit-distance fallback in `replace_most_similar_chunk` sits after a bare `return` and never runs; zero MCP references. No corrections.

In One Paragraph

aider makes one model call per turn and then runs a deterministic pipeline: parse the model's prose for edit blocks, apply them, lint and test, and send any errors back. There are no tool calls at all. Where pi is model-agnostic, aider is model-aware: a 313-entry compatibility table decides which edit format each model gets, a PageRank-ranked repository map shows the model the codebase, and git is the undo system.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

aider is a negotiator of text formats. The model never "calls" anything. It writes SEARCH/REPLACE blocks, unified diffs, or whole files inside its normal reply, and aider parses that text. This design dates from before function calling was common, and it still means aider works with any model that can follow formatting instructions.

The price is that every format is a fragile contract that must be tuned per model. That is what the 313-entry table and dozens of substring rules (such as "if `gpt-4` is in the model name, use this format") are for. pi's thesis was "mechanism in the core, policy in extensions." aider's is closer to "the model writes, aider parses, verifies, and remembers which models need which format."

A Turn, Step by Step

The user types "add error handling to foo.py" with `foo.py` already added to the chat.

1. **Input.** The REPL reads the line, checks for slash commands, looks for file names mentioned in the text, and detects URLs.
2. **Prompt assembly.** The message order is fixed: system prompt, examples, read-only files, the repo map, summarized history, the full contents of chat files, the user's message, then a reminder. The repo map and file contents are sent as *fake conversation* — a user message followed by an assistant message such as "Ok, I won't try and edit those files without asking first." Prompt-cache breakpoints are placed on the stable sections, and a background thread can keep the cache warm with one-token calls every five minutes.
3. **Model call.** One call through LiteLLM, streamed. No function calling.
4. **Parsing.** aider scans the reply for SEARCH/REPLACE fences and works out the target file from the three preceding lines (exact match, then base name, then fuzzy match). Shell code blocks are collected separately and never run inline.
5. **Applying edits.** A dry run, then the real write. Each block is matched exactly, then with flexible leading whitespace, then with `...` elision. A block that matches nothing produces an error message with "did you mean" lines, and that message becomes the next prompt.
6. **Commit.** Every applied turn becomes a git commit with a model-written message. The commit hash is recorded so `/undo` can revert only aider's own commits.
7. **Verify and reflect.** If auto-lint is on, aider lints the edited files and asks "Attempt to fix lint errors?" A yes sends the lint output back as a new message. Suggested shell commands each need their own explicit confirmation — neither "All" nor `--yes-always` can approve them. Auto-test follows the same pattern.
8. **Rendering.** Output streams as markdown; afterward aider prints what it applied and an `/undo` hint.

The loop has three levels: the outer REPL, a **reflection loop** that re-prompts with errors (capped at three), and a **retry loop** with exponential backoff for API errors.

Subsystems Worth Knowing

- **Repo map.** The signature feature. Tree-sitter extracts definitions and references from every file (cached in SQLite). A graph of files is built, with edges weighted by how identifiers are used — mentioned identifiers weigh ten times more, private names a tenth as much, references from chat files fifty times more. Personalized PageRank ranks the result. A binary search then fits as many ranked symbols as possible into a token budget

(by default one-eighth of the model's input window, clamped between 1,024 and 4,096 tokens).

- **Edit formats.** Each format is a `Coder` subclass: `diff`, `udiff`, `whole`, `patch`, `ask`, `architect`, `context`. Switching format rebuilds the coder and summarizes history, because old replies in a different format would confuse the model.
- **Architect mode.** The nearest thing to sub-agents: one model plans, a second model edits, with a user confirmation in between.
- **History.** The live turn and past history are separate lists. Old turns are summarized in a background thread by the cheaper "weak" model. The chat log is a markdown file in the repo.
- **Watch mode.** A file watcher looks for `AI!` and `AI?` comments in your code and turns them into prompts. The filesystem becomes an input device.

Design Tradeoffs

- **No doom loop to worry about.** Each turn is exactly one model call plus a deterministic pipeline, with at most three reflections. The dangerous unbounded loop of tool-calling harnesses cannot happen here.
- **Model awareness has a maintenance tax.** The 1,323-line model file grows by accretion. Some branches are already dead (an earlier `gpt-4` check returns first), and the fuzzy-match fallback is unreachable.
- **Configuration, not code.** There is no plugin API, no hooks, no MCP, no skills. Extending aider means changing settings, choosing a format, or forking. That is a direct consequence of the text-format bet: with prose as the contract, there is less machinery to plug into.
- **Re-send and cache versus diff.** aider rebuilds the full prompt every turn and leans on prompt caching; pi sends deltas. Same problem, opposite solutions.

Security Lens

- **Edits to chat files apply without asking.** Prompts appear only at the edges: creating a new file, editing a file not in the chat, and running any shell command. The safety net is git, not approval.
- **Shell commands are the strongest gate in the product.** Each suggested command needs its own yes, and bulk-approval flags cannot bypass it. This is worth noting as a good pattern.
- **No sandbox of any kind.** Anything the model gets past a human confirmation runs as the user.
- **Prompt-injection surface.** The repo map, read-only files, web scraping (`/web`), clipboard watching, and watch mode all pull outside text into the prompt. With no tool calls, injected text can only influence edits and suggested commands, both of which pass through git or a confirmation — a smaller blast radius than tool-calling harnesses.
- **Telemetry exists** (Mixpanel and PostHog); check settings in sensitive environments.

Take-away

The repo map is the idea to steal: an algorithmic, token-budgeted summary of the whole codebase, re-personalized every turn so the model can navigate without being told where to look.

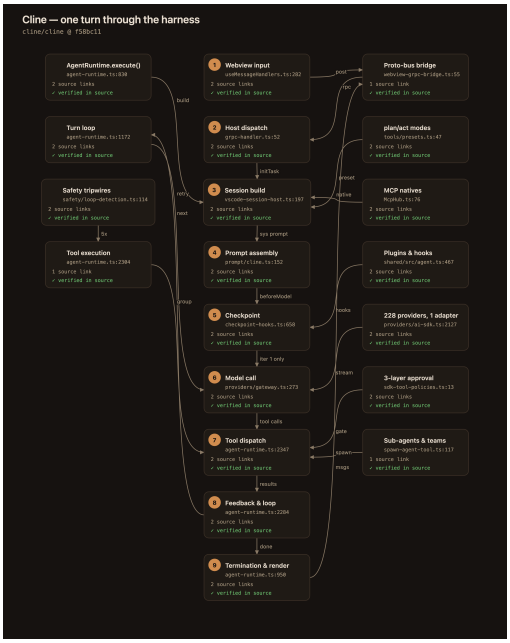
3. Cline — The Approval-Gated IDE Resident

Repo: cline/cline · Pinned: v4.1.22 → f58bc11 (Sep 29, 2026) · TypeScript (bun workspace) · Apache-2.0 · Scope: the VS Code extension

> **Verified.** The proto directory defines 22 services (16 `cline`, 6 `host`) with 224 RPCs; the VS Code session factory sets `maxIterations: undefined`; loop detection uses a soft threshold of 3 and a hard threshold of 5; checkpoints use `git stash create` under `refs/cline/checkpoints/`; the docs still describe a separate "shadow Git repository." No corrections.

In One Paragraph

Cline's agent loop is a host-independent SDK. The VS Code extension is one host; the webview UI talks to it over a generated bridge of 22 services and 224 RPCs, never sharing memory with the runtime. Every tool call passes a three-layer approval model with diff previews, and every user turn is snapshotted with `git stash` so `/undo` can restore the worktree.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

Cline makes policy the product. Where pi pushes policy to extensions and aider replaces it with git, Cline asks — and makes asking cheap (one click, with a diff preview) and undoing certain (per-turn snapshots). The user is the sandbox.

Architecturally, the loop (`@cline/agents`, about 2,900 lines) knows nothing about VS Code. It emits typed events and accepts a bag of seven hook callbacks. The VS Code host supplies configuration: which tools exist, how they execute, the approval callback, and the checkpoint hook. The command-line host runs the same loop with the opposite policy — everything auto-approved. The loop is the product; every interface is a host.

A Turn, Step by Step

The user types "add a retry helper to src/utils.ts and run the tests" in act mode with default settings.

1. **Webview.** The React UI calls a generated `newTask` RPC, which is posted to the extension host as a message.
2. **Host dispatch.** A generated handler table routes the call to the session controller.
3. **Session build.** Act mode selects seven built-in tools. One quiet substitution happens here: VS Code removes the SDK's shell tool and registers its own terminal-backed `run_commands` under the identical name — same policy key, different execution, invisible to the policy layer. If the model ID contains `gpt` or `codex`, the `editor` tool is swapped for `apply_patch`.
4. **Prompt assembly.** A template is filled with platform, date, IDE name, working directory, user rules, and metadata, then the message list is normalized.
5. **Checkpoint.** On the first iteration of each user turn only, a `beforeModel` hook snapshots the worktree with `git stash create`, plus a synthesized commit for untracked files, stored under a private ref *in the user's own repository*. A separate index file means the user's staging area is never touched.
6. **Model call.** The gateway resolves the model by exact ID against a generated catalog and streams through the Vercel AI SDK.
7. **Tool dispatch and approval.** Reads and edits are auto-approved by default. When the model calls `run_commands`, the webview prompts. Edits get a diff preview rendered from a virtual document. A denial returns "The user denied this edit. The file was NOT modified…" so the model does not assume the change happened.
8. **Termination and rendering.** When the model stops calling tools, a finish event is translated into UI messages and streamed to the webview, where a reducer converges the transcript even if messages arrive out of order or twice.

Subsystems Worth Knowing

- **The three approval layers.** (1) The SDK gate: tool policies default to *enabled* and *auto-approve*, so unlisted tools fail open; a missing approval callback means auto-deny. (2) VS Code forces `autoApprove: false` on every governed tool so the SDK always consults the live UI settings. (3) The approval callback applies live settings, shows diffs, and waits for the user.

- **Loop limits.** No turn cap in the VS Code host. Instead: a nudge after 3 identical consecutive tool calls, an abort at 5, and a stop after 6 consecutive failed turns. Output-token cut-offs get 3 recovery retries.
- **Checkpoints.** One per user turn, restorable with `/undo`. Restore refuses if HEAD moved, and takes a safety snapshot before resetting.
- **Providers.** 228 provider specs — 211 generated from models.dev plus 17 hand-written — behind *one* generic adapter. The generated catalog is almost 200,000 lines.
- **MCP.** Servers from a watched settings file (stdio, SSE, streamable HTTP). Their tools become native tools named `server__tool`.
- **Hooks and plugins.** The seven-callback runtime hook bag, a plugin interface that returns tools and hooks, and ten file-based user hook scripts. One named hook (`PreCompact`) is not wired.
- **Modes.** Only plan and act in VS Code, and they differ by exactly one tool (`editor` is off in plan). Sub-agents and agent teams exist in the SDK but are disabled in the VS Code host.

Design Tradeoffs

- **Interruptions versus surprises.** Asking constantly causes prompt fatigue, so defaults approve reads and edits and ask only about commands. The single command toggle is named "safe commands," but turning it on approves *every* command — there is no notion of a safe command in the code.
- **Private stash refs versus public commits.** Cheaper and less noisy than aider's commits, but harder to discover — and the docs still describe the old shadow-repo design.
- **SDK loop versus embedded loop.** The CLI reuses the whole loop with a one-line policy change. The cost is a real distributed-systems problem between host and webview.
- **Generated catalog versus hand-tuned table.** Almost no brittle string matching, but a huge catalog that must be regenerated as the model world shifts.

Security Lens

- **Fail-open default in the SDK.** Any tool without an explicit policy is enabled and auto-approved. The VS Code host compensates by setting policies on every governed tool — so anything that escapes that list escapes the gate.
- **MCP naming seam.** Policies are keyed on the raw `server__tool` name, but registration rewrites any name longer than 64 characters or containing characters outside letters, digits, `_` and `-` (a dot or space in a server name is enough). The rewritten tool misses its "ask" policy and falls through to the default auto-approve — bypassing even the global MCP toggle. This is the single most interesting seam in the chapter for an assessor.
- **Per-tool MCP approval lists are dead.** Only the global MCP toggle is honored on the SDK path.

- **Commands run unsandboxed** in the user's shell. The plan-mode command guard is self-described as "a simple blacklist, not a shell interpreter," and will not catch something like `python -c "open(...,'w')"`.
- **Loop detection is reactive.** It is fed from the tool-started event, so the fifth identical call has already started when the abort lands.

Take-away

Separate the loop from the host. Once the loop is a library with a hook bag, a second host with an opposite policy costs one line — and the security posture becomes explicit configuration you can audit.

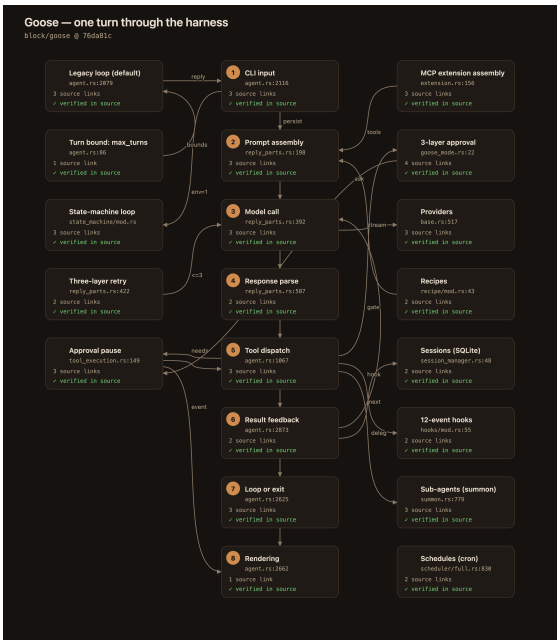
4. Goose — The MCP-Native Agent That Asks a Judge

Repo: block/goose · Pinned: v1.53.0 → 76da81c (Sep 30, 2026) · Rust + Electron · Apache-2.0 · Scope: the CLI agent

> **Verified.** `DEFAULT_MAX_TURNS = 1000`; the state-machine loop turns on only when `GOOSE_STATE_MACHINE` is `1`, `true`, `TRUE`, or `yes`; the legacy `agent.rs` is 6,382 lines; the SmartApprove judge labels requests "UNTRUSTED TOOL REQUEST DATA"; modes are Auto, Approve, SmartApprove, Chat; exactly four extension types, none of them SSE; the repetition inspector is registered with `None`; 12 hook events; the repo's AGENTS.md requires changes in both loops. No corrections.

In One Paragraph

In Goose, every tool is an MCP server — including the built-in ones, which run as in-process MCP over an in-memory channel. Tool calls pass a three-layer approval model whose SmartApprove mode asks a language model to judge whether each call is read-only. The codebase is mid-migration from a legacy `loop {}` to an effect-sourced state machine. Recipes turn prompts into runnable, schedulable programs.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

Make everything a server, and the agent becomes a client of its own capabilities. There is no non-MCP tool path in Goose at all; the agent's toolset is whatever extension configs say it is for that session. The tool contract is a *protocol*, not an API.

The second thing to hold in mind is the migration. The CLI runs the old loop by default. The new loop is an ordered pipeline of 18 operations where the model call comes *dead last*: entry hooks, slash commands, steering, the turn limit, shell bangs, compaction, approval, skills, recipes, tool execution, retries, stop hooks — and only then inference. Every policy decision is an auditable effect on persisted state before the model is consulted. Until parity is declared, every behavior change must land in both loops.

A Turn, Step by Step

The user types "list the files in src/ and summarize what this project does" in Auto mode.

1. **Input.** The message is assigned an ID, hooks for session start and prompt submission fire, slash-command handling declines, and the message is saved to SQLite.
2. **Prompt assembly.** Tools are collected from every extension, names are checked for uniqueness, and the system prompt is built from extension info, directory hints, and the current mode. A turn-context message (time, directory, compaction status, turn budget) is added as an agent-only message.
3. **Model call.** Iteration 1 of up to 1,000. The provider streams; transient errors before the first token are retried up to three times. Mid-stream errors are not retried.
4. **Parsing.** Tool requests are extracted, model-mangled names are repaired, arguments are coerced to the advertised schema, and duplicate call IDs removed. Calls to tools that were never advertised become error results — invalid calls are data, not exceptions.
5. **Approval and dispatch.** Each request is sorted into approved, needs-approval, or denied. A read-only `ls` is approved and dispatched to the owning extension. A mutating call would register a one-shot channel, emit an "action required" event, and *pause the loop mid-iteration* until the user answers.
6. **Feedback.** Results go back as tool-response blocks. Large outputs (over 200,000 characters by default) spill to an owner-only temp file.
7. **Exit.** On iteration 2 the model answers in text, the stop hook allows it, and the loop breaks.

Subsystems Worth Knowing

- **Extensions.** Four kinds: `stdio` (a subprocess, with an OSV malware check on `npx`/`uvx` launches), `builtin` (in-process MCP), `platform` (in-process, created by the client factory), and `streamable_http` (remote, with OAuth). Tools are namespaced `extension__tool`, but true ownership is a `_meta` tag.
- **The approval model.** Three layers. *Modes*: Auto, Approve (ask everything), SmartApprove (ask only for sensitive calls), Chat (no tools). *Permission levels* per tool in `permission.yaml`: NeverAllow beats AlwaysAllow beats AskBefore. *Inspectors* in order: Security, Egress, Adversary (user-written rules reviewed by a model), Permission, Repetition. Security can only make a decision stricter, never looser.

- **The SmartApprove judge.** For tools with no cached verdict, Goose asks the model whether each pending call is read-only, presenting the calls as untrusted data. Non-read-only verdicts are cached as AskBefore. If the judge fails, everything prompts.
- **Providers.** One trait with a single required method (`stream`); 36 statically registered providers, 48 declarative JSON providers, and user-defined ones. An 8,160-entry model catalog is bundled at build time and refreshed at runtime.
- **Sessions and compaction.** SQLite. The full recipe is stored on the session so a resumed session replays the same task. Compaction triggers at 80% of the context window.
- **Hooks.** Twelve lifecycle events loaded from plugin `hooks.json` files; shell commands receive JSON on stdin and block with exit code 2.
- **Recipes.** Versioned YAML task files with instructions, a first prompt, their own extensions, model settings, typed parameters, and an optional response schema that forces a `final_output` tool. They can be scheduled with cron; scheduled runs always use Auto mode.
- **Sub-agents.** A `delegate` tool runs a full in-process agent with its own session. Sub-agents are forced into Auto mode because approvals cannot yet be forwarded to the parent, and they cannot nest.

Design Tradeoffs

- **Protocol everywhere.** Uniform, but every tool call pays MCP serialization, and the built-in toolset is a config list you can break by editing YAML.
- **Adaptive approvals.** The judge is smarter than a static list — and it is a model guarding against a model, which the in-tree prompt-injection tests acknowledge.
- **All three model-knowledge strategies at once.** A big generated catalog, aider-style substring heuristics, and many static providers. There is no validation of model names anywhere.
- **No undo.** Safety is entirely preventive: modes, inspectors, the judge, hooks. If the agent breaks your tree, your backups are your own.

Security Lens

- **The doom-loop guard is theater.** The repetition inspector is registered with no limit, so it always allows. The real bound is the 1,000-turn cap.
- **Auto is the default mode,** and headless Auto auto-allows with only a warning in the log. Approve and SmartApprove in non-interactive mode are hard errors — a good, strict choice.
- **Forced Auto in sub-agents and schedules.** Anything delegated or scheduled runs without prompts. An attacker who can influence a recipe or a delegation target gets unattended tool use.
- **"Always Allow" is withheld when a security inspector raised a warning.** A good pattern worth copying.

- **Supply-chain edge.** The OSV check runs when an `npx`/`uvx` extension is spawned and fails open for unknown ecosystems. Shell commands run unsandboxed; in a Flatpak build they are routed to the host.
- **Recipe hygiene.** Recipes are scanned for Unicode tag characters (a hidden-instruction trick), and `file` parameters cannot have defaults so a shared recipe cannot quietly import a sensitive file.

Take-away

Put the model call last. Goose's state machine shows how to make every hook, compaction, and approval an ordered, persisted operation that happens before inference — the most auditable loop design in the series.

5. Codex CLI — The Sandbox-First Agent That Speaks One Wire Protocol

Repo: openai/codex · Pinned: rust-v0.160.0 → a956835 (Oct 1, 2026) · Rust, 154-crate workspace · Apache-2.0 · Scope: the Rust CLI

> **Verified.** The workspace has 154 members; the only `max_turns` symbol outside tests is a TUI recap constant; `WireApi` has one variant, `Responses`, and `"chat"` is rejected with a dedicated error; `codex exec` starts an in-process app-server client; the approval code's own comment says precedence is "1. Hooks" then Guardian or user; the Linux sandbox uses bubblewrap, landlock, and seccomp.
>
> **Correction (C2).** The site's comparison matrix lists 14 `RolloutItem` variants. The code has 12, which is what the Codex article body says.

In One Paragraph

Every Codex interface — the terminal UI, non-interactive `exec`, and the background daemon — is a client of a single JSON-RPC app-server: one protocol, one loop. The turn logic has three nested loops and no turn cap anywhere in the tree. Tool calls go down a four-layer sandbox stack (on Linux: bubblewrap, landlock, and seccomp) and an approval chain in which hooks outrank the user.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

Constrain the blast radius in the operating system, and you can afford to let the model act. Codex has the heaviest safety machinery in the series — kernel-level confinement, not shell wrappers — and pairs it with the narrowest wire protocol: only OpenAI's Responses API, no Chat Completions path at all.

The second idea is that the loop lives behind a server. Even `codex exec`, the scriptable one-shot mode, does not call the core library directly; it starts an in-process app-server and drives it over JSON-RPC. The upside is uniformity: every frontend speaks the same protocol, including headless JSON output. The cost is a protocol boundary inside what used to be a function call.

A Turn, Step by Step

The user types "refactor the retry helper to use exponential backoff" in the TUI.

1. **Submit.** The TUI sends a `turn/start` RPC with the text plus sandbox and permission overrides from config.
2. **Route to core.** The app-server converts it to an operation and starts a task with a fresh cancellation token.
3. **Task loop.** The outer task loop re-runs whole turns while pending user input remains.
4. **Turn preamble.** Guardian check, compaction if needed, MCP setup, skill and plugin injection, session-start hooks; the user message is recorded.
5. **Prompt.** Each iteration re-resolves tools and permissions, then builds a request with `parallel_tool_calls: true`, `store: false`, and a thread-scoped prompt-cache key.
6. **Model call.** WebSocket first, falling back permanently to HTTP/SSE, both to `/responses`.
7. **Stream handling.** Text streams to the TUI. Each completed tool call becomes a future and runs concurrently.
8. **Approval, parked inside the tool.** A gated call runs the chain — hooks, then the Guardian reviewer, then the user — while the turn loop waits for all in-flight tools to finish. A denial becomes a failed tool result, not a loop exit.
9. **Loop or stop.** If any tool ran, the next iteration rebuilds the prompt from history. When the model replies with only a message, stop hooks and post-turn compaction run and the turn ends.

Mid-stream user input cancels the current step and is folded into the next iteration — Codex's form of steering.

The Sandbox Stack

Four layers, each with its own language and failure modes:

1. **Exec-policy language.** A Starlark rule language decides *allow, prompt, or forbid* before anything runs. It has three built-in rule types: command prefixes, network rules, and host-executable pinning (a defense against PATH spoofing). Decisions combine by taking

the strictest. No default rules file ships, so built-in heuristics decide until the user adds rules; approving "always allow" appends a rule.

2. **Permission profile.** The declarative *what*: managed filesystem and network access, disabled, or delegated to an external caller. The familiar names `read-only`, `workspace-write`, and `danger-full-access` survive only as diagnostic labels, and the code says they "must not be used for authorization."
3. **Platform backends.** macOS uses Seatbelt profiles starting from deny-all. Linux uses bubblewrap (read-only root with writable roots layered on, new user and PID namespaces, no network when restricted) plus landlock and seccomp. Windows has three backends with different capabilities.
4. **Network proxy.** A local man-in-the-middle-capable proxy is the only route to the internet in restricted modes, which is how macOS gets per-domain control despite Seatbelt's on/off network switch.

Escalation is kept separate: a declarative "run outside the sandbox" flag, a "retry outside the sandbox?" prompt after a sandbox denial, and a patched shell that asks per `exec()`.

Subsystems Worth Knowing

- **Per-action approvals.** A sealed list of approvable actions: run a command, write to stdin, exec, apply a patch, call an MCP tool, network access, request permissions. Writing to a running process's stdin is its own category.
- **Approval ladder.** UnlessTrusted, OnRequest (the default), Granular, Never. Many review decisions *amend policy* — approving a command for good writes Starlark to the user's rules file.
- **Tools.** No built-in read-file, write-file, edit, or search tool. Reads go through `exec_command`; edits go through `apply_patch`. Heavy tool namespaces are deferred and loaded through `tool_search`.
- **Providers.** Five built in (OpenAI, two Bedrock variants, Ollama, LM Studio), eight authentication modes; the repo refuses to bundle third-party providers.
- **Sessions and memory.** Rollout JSONL files by date, mirrored to SQLite, with resume and three fork modes. A feature-gated pipeline extracts cross-session memories with a model at startup.
- **Hooks, skills, plugins.** 12 hook events (shell commands or MCP servers) that can block, rewrite input, or force approval decisions. Skills are prompt injection, not tools. Slash commands are a closed list compiled into the TUI.
- **Sub-agents.** Two generations selectable by config, depth-limited.

Design Tradeoffs

- **No `read_file` is the point.** When every read is a subprocess, the sandbox sees everything. A native file-read tool would bypass the Starlark layer entirely.

- **Freedom of iteration, constraint of action.** The most permissive loop in the series (no turn cap, no repeated-call detector) sits on the most restrictive execution environment.
- **One wire protocol.** Total control of prompt shape and caching, but any provider that does not speak Responses needs a shim.
- **Three representations of "how locked down."** The runtime profile decides; a legacy config enum and diagnostic labels describe. That is a migration seam to watch.

Security Lens

- **Hooks sit above the human.** They run first in the approval chain and can force decisions. Whoever controls hook configuration controls approvals — treat hook config as privileged.
- **Malformed rules fail open.** A syntax error in the user's rules file falls back to the requirements policy with a warning. Heuristics then decide unmatched commands — a second, implicit authority.
- **Model-specific rule stripping.** A setting can silently strip user allow rules for certain models.
- **Guardian is a model in the loop.** A "no opinion" from the reviewer falls through to the user ("no contributor is never an implicit allow") — the right default.
- **Escape hatch exists.** `--dangerously-bypass-approvals-and-sandbox` turns everything off in `exec`. Check CI configurations for it.
- **Platform asymmetry.** Windows' restricted-token backend cannot enforce deny-reads; the code raises hard errors instead of pretending. Assess per OS.

Take-away

If the model must shell out to see the world, make the shell the choke point. Codex shows that removing convenience tools can be a security feature when it forces every action through one well-guarded path.

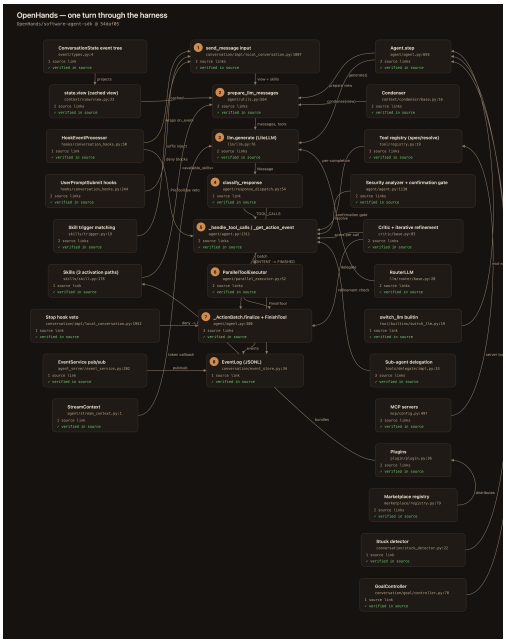
6. OpenHands — The Event-Sourced Agent SDK

Repo: OpenHands/software-agent-sdk · Pinned: v1.53.0 → 54daf05 (Oct 5, 2026) · Python (uv workspace) · MIT

> **Verified.** `max_iteration_per_run` defaults to 500; exactly five event types (action, observation, message, system prompt, agent error); six hook events; a hook failure returns ALLOW; `RouterLLM` with multimodal and random routers; `switch_llm` and `classify_and_switch_llm` built-in tools; no OS-sandbox code in the SDK or tools packages. The `OpenHands/OpenHands` repo's README now describes Agent Canvas, and the agent lives in `software-agent-sdk`, as the article says.
>
> **Correction (C6).** The landing page says microagents "are dead." The SDK still scans `~/.openhands/microagents/` as legacy support and converts what it finds into Skills. They are deprecated, not removed.

In One Paragraph

In OpenHands, everything that happens is a typed event in a persistent tree, and the model never sees the raw stream — only a cached view that enforces rules about what it may see. Hooks intercept the event stream, a critic can refuse to let the agent finish, and model routing can change which model answers each call, including the agent switching its own model mid-run. The older "microagents" have been replaced by skills in the open AgentSkills format.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

Make the event log the substrate and let the loop be dull. User messages, tool calls, tool results, errors, hook decisions, condensation, and interrupts are all events. One event system serves the loop, the hooks, persistence, streaming, and the server that fans events out to web clients. Everything speaks the same language — at the cost that nothing is a direct function call.

A scope note matters here. If you look for the agent at `OpenHands/OpenHands`, you find Agent Canvas, a control center for running *other* agents. The organization was renamed from All-Hands-AI to OpenHands, and the harness moved to `software-agent-sdk`.

A Turn, Step by Step

The user sends "fix the failing test in tests/test_auth.py."

1. **Input.** `send_message` resets a finished or stuck conversation, runs `UserPromptSubmit` hooks (a deny blocks the message entirely), matches skill triggers, and emits the user message event.
2. **Prompt assembly.** Model input is built from the cached view, never the raw list. The system prompt lists every available skill (name, description, location) so the model can read one on demand.
3. **Model call.** A LiteLLM wrapper sends the request, choosing Chat Completions or the Responses API per model, and asks the model to label each action's security risk.
4. **Classification.** A pure function classifies the reply: tool calls, content, reasoning only, or empty. Reasoning-only and empty replies get a corrective nudge.
5. **Dispatch.** Each call is parsed, validated with Pydantic, risk-scored, and optionally scored by the critic, then emitted as an action event. As it passes the hook processor, `PreToolUse` hooks can block it. Then the confirmation policy decides; if confirmation is needed, the loop stops and waits.
6. **Feedback.** The batch is cut at any `finish` call, hook-blocked actions get rejection observations, and the rest run in parallel threads. Observations are emitted in the original call order.
7. **Termination.** When the model calls `finish`, the critic may score the work below threshold and inject a follow-up prompt instead of finishing. Then Stop hooks get a vote and can send the agent back to work.
8. **Rendering.** Every event is appended to a JSONL log and, in server mode, published to REST and WebSocket clients.

Subsystems Worth Knowing

- **The view.** An incrementally maintained projection of the event stream with enforced rules: each observation appears once, tool calls match results, batches stay atomic.
- **The critic.** Scores work from events and an optional git patch. With iterative refinement on, "done" means the model *and* the critic agree.

- **Security.** Three cooperating parts: model self-labels, an authoritative analyzer (LLM, deterministic rails, ensemble, or a third-party service), and a confirmation policy (always, never, or only above a risk threshold). Without an analyzer, risk is always "unknown" and the model's own label is ignored.
- **Tools.** A tool in the SDK is a *spec* resolved to an implementation at runtime; the SDK never imports the tools package. Defaults: terminal, file editor (with per-file undo), task tracker.
- **Hooks.** Six events, three executor types: a shell command, a model prompt acting as judge, or a full sub-agent.
- **Condensers.** Pluggable context compaction, triggered proactively or after a context-window error.
- **Stuck detection.** Repeating action-observation loops, error streaks, monologues, and alternating patterns move the conversation to STUCK.
- **ACP host mode.** An agent class that hands the *entire loop* to an external agent such as Claude Code, Codex, or Gemini CLI. Here the harness hosts other harnesses.

Design Tradeoffs

- **Uniformity versus indirection.** Every stage is observable and interceptable because it passes through the stream — and a slow consumer or misbehaving hook sits in line with the loop.
- **Judging your own homework.** A harsh critic and a compliant model can orbit each other, bounded only by the 500-iteration cap and the budget.
- **SDK, not product.** No git safety net, no OS sandbox in the loop, no scheduler. Those are the deployer's job; Docker sandboxes come from the agent server.

Security Lens

- **Hooks fail open.** A crashed hook returns ALLOW so it never blocks the loop. That is a deliberate availability choice; it means a hook is not a reliable security control unless you make its failure mode explicit.
- **Async `PreToolUse` hooks cannot block** — they warn and run in the background.
- **Security depends on configuration.** With no analyzer configured, every action's risk is "unknown," and a "confirm risky" policy has nothing to act on. Verify the analyzer and confirmation policy in any deployment you assess.
- **Agent-swapped models.** The agent can switch its own model with a tool call. If models differ in safety tuning or cost, that is a lever worth reviewing.
- **Skills can carry tools and servers.** Repository skills can bundle MCP servers and declare pre-approved tools — an important trust boundary when opening untrusted repos.
- **No sandbox in the SDK itself.** Isolation is provided by the agent server's Docker runtimes; a local workspace is just a process.

Take-away

Never let the model read the raw log. A maintained view with enforced invariants is a clean place to guarantee what the model can and cannot see — and to fix malformed history before it reaches the provider.

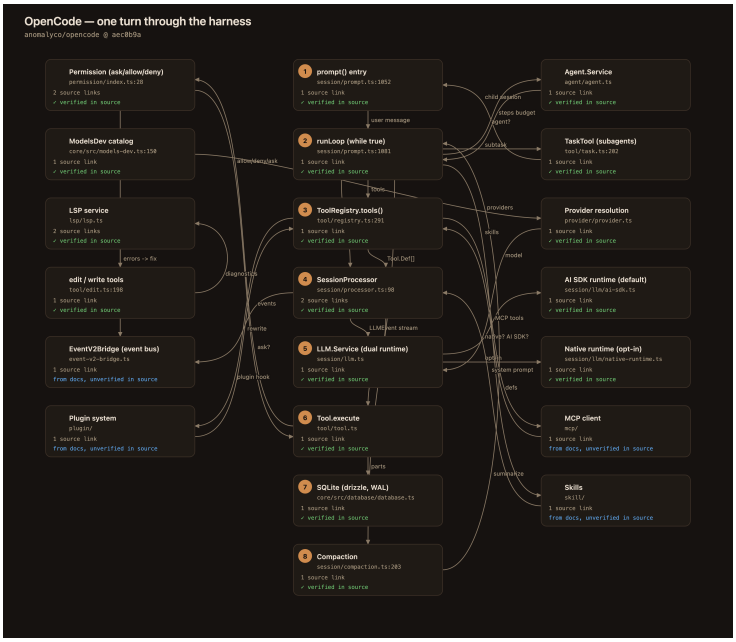
7. OpenCode — The Dual-Runtime Coding Agent

Repo: anomalyco/opencode · Pinned: v1.18.34 → aec0b9a (Sep 30, 2026) · TypeScript on Bun, Effect-based · MIT · About 212,000 GitHub stars on Oct 12, 2026

> **Verified.** The native transport turns on with `OPENCODE_EXPERIMENTAL_NATIVE_LLM`; the tool registry swaps `apply_patch` in for `edit` and `write` when `usePatch` is true; LSP diagnostics are capped at 20 errors per file; `maxSteps` defaults to `Infinity`; permission evaluation uses `findLast` (last matching rule wins); Effect is imported in 227 source files; the star count (over 210,000) holds.
>
> **Correction (C5).** The landing page says OpenCode "runs its session loop on two runtimes." There is one session loop. Underneath it are two LLM transports — the Vercel AI SDK by default and an opt-in native client — which both produce the same event stream. The article describes this correctly.

In One Paragraph

OpenCode is built on Effect, a typed functional-effects library for TypeScript, which supplies its dependency injection. One session loop sits on top of two interchangeable LLM transports. The tool set is rebuilt on every request so the model sees only the tools that suit it. Permissions are decided at the moment a tool is used, and compiler errors come back *inside* the tool result through the language server, not through a separate repair loop.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

Keep the loop dumb and put the intelligence in the seams. The loop is about 80 lines: spin forever, re-read the session's messages from SQLite each time, and exit only when the last assistant message finished for a reason other than tool calls, no tool calls are pending, and that message is a direct reply to the user's message. A "stop" with tool calls present does not exit.

Effect shapes everything: services are typed tags composed into layers, errors are typed values, and the agent loop is an Effect program. That gives OpenCode cancellation and resource safety, and it gives the permission system a natural way to *park* a tool call until a user answers.

A Turn, Step by Step

The user sends "Add a retry loop to the fetch call in src/api.ts."

1. **Input.** The prompt is saved and the turn runs inside a per-session guard, so concurrent prompts line up rather than interleave.
2. **Loop.** Each iteration filters out compacted messages, checks the three-part exit condition, then drains a task queue (sub-tasks become child sessions; context overflow triggers compaction).
3. **Tools.** The registry builds the tool set for this exact model. GPT-family models get `apply_patch` *instead of* `edit` and `write`. Plugins can rewrite any tool's description or parameters through a `tool.definition` hook. MCP and plugin tools join dynamically.
4. **Stream.** Before the model call, the processor snapshots the working tree, because the AI SDK may execute tools before emitting events.
5. **Model.** The request goes to the default transport or the native one; both emit the same `LLMEvent` stream, so the processor never knows which ran. Model details come from a live catalog fetched from models.opencode.ai and cached locally.
6. **Execute.** Each tool call hits the permission gate. Rules from the session, prior approvals, and agent config are layered, and the last matching rule wins, defaulting to "ask." An ask publishes an event and waits. After an edit, the tool appends any LSP errors to its own output: "LSP errors detected in this file, please fix."
7. **Persist.** Every message and part is written to SQLite in WAL mode. The loop holds no conversation state in memory between iterations.
8. **Compact.** Each iteration checks for context overflow. If needed, the head of the conversation is summarized and a recent tail kept; pruning runs in the background after the loop.

Subsystems Worth Knowing

- **Model-adaptive tools.** The tool set is a negotiated surface. The editing affordance itself changes by model family.

- **Permissions.** Last-matching-rule-wins with default "ask." Answering "always" adds a rule for the rest of the session. On shutdown, pending requests are rejected rather than left hanging.
- **LSP feedback.** Errors only (warnings stay quiet), at most 20 per file. The compiler loop closes inside the tool result, so the model fixes errors on its next turn without any harness retry logic.
- **No hard step cap.** `maxSteps` defaults to infinity. When a configured step limit is reached, OpenCode does not kill the run; it injects a prompt asking the model to wrap up.
- **Sub-agents.** Full child sessions with their own history and loop, spawned by a `task` tool. Background tasks report back as synthetic user messages.
- **Event bus.** A session-scoped publish/subscribe channel that the TUI, `opencode serve`, and ACP all subscribe to.

Design Tradeoffs

- **Per-request computation.** Rebuilding tools and resolving permissions each time costs work on every turn, in exchange for fitting the model exactly.
- **Live catalog.** New models work without a release, but the harness cannot start offline with full provider knowledge.
- **Stateless loop.** Re-reading from SQLite each iteration makes interrupts, compaction, and multiple frontends safe, at the cost of more database work.
- **Trusting the model to stop.** No iteration cap unless configured.

Security Lens

- **Order matters in last-match-wins.** A broad "allow" added later can override an earlier "deny." When assessing a configuration, read rules bottom-up.
- **"Always" learns within the session.** One permissive click widens the policy for every later call in that session.
- **Plugins can rewrite tool descriptions.** The `tool.definition` hook changes what the model believes a tool does — powerful, and a place where a malicious plugin could mislead the model.
- **No OS sandbox in the loop.** The permission gate is the only control on tool execution.
- **Network dependency at startup.** The provider catalog is fetched from a remote endpoint; in locked-down environments, check the local cache and the `OPENCODE_MODELS_URL` variable, which points the catalog at a different source.
- **Product-gated tools.** Web search is available only with OpenCode's own provider, a reminder that the tool set is also a business surface.

Take-away

Close the feedback loop inside the tool. Appending compiler diagnostics to an edit's result costs almost nothing and removes the need for a separate repair loop.

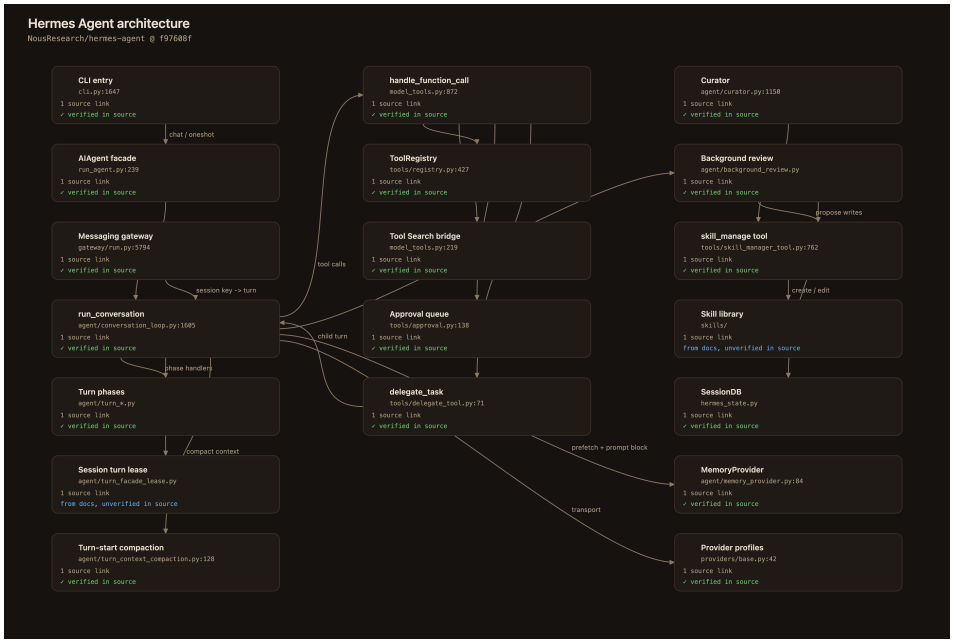
8. Hermes Agent — The Self-Improving Agent

Repo: NousResearch/hermes-agent · Pinned: v2026.9.24 → f97608f (Sep 24, 2026) · Python, with a TypeScript TUI · MIT · About 250,000 GitHub stars on Oct 12, 2026

> **Verified.** The curator's default interval is 7 days (with a 2-hour minimum idle time), and it archives but never deletes; `_spawn_background_review` exists and forks a review after turns; a Tool Search bridge collapses large catalogs; delegated sub-agents default to 250 iterations; the star count (over 250,000) holds.
>
> **Correction (C3).** The article says `max_iterations` defaults to 500. In code, the agent class defaults to unlimited (`sys.maxsize`); the CLI's built-in config sets `agent.max_turns: 500`; and the messaging gateway falls back to unlimited when no limit is configured. Treat 500 as the CLI default, not a universal one.
>
> **Wording note.** The landing page says a curator "promotes" new skills. In code the curator pins, archives, consolidates, and patches skills. "Promote" is best read as "pin."

In One Paragraph

Hermes writes its own skills while it runs. After a turn, a background review can fork the agent to ask whether anything it just did should be saved as a skill or a memory. On a much slower clock, a curator wakes during idle time to pin, archive, or merge skills. The harness learns from sessions without a person editing its prompts.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

Hermes is a multi-tenant agent with a learning loop bolted onto a conventional synchronous core. One gateway process multiplexes users from Telegram, Discord, WhatsApp, Slack, and Signal into separate session namespaces. Each session runs the same turn: take a lease, compact preemptively, call the model repeatedly, dispatch tools through a single funnel, save to SQLite — and then, the distinctive part, maybe spend another model call deciding what the turn taught it.

The bet is that an agent that maintains its own skills compounds over time, and that the compounding is worth the extra tokens.

A Turn, Step by Step

The user sends "Summarize what we decided about the API rate limits yesterday."

1. **Input.** From the CLI or a messaging gateway; the gateway maps the platform user, chat, and thread to a session key.
2. **Lease.** A per-session lease means concurrent prompts line up instead of interleaving.
3. **Preflight.** Compaction runs *before* any model call: make room first.
4. **Prompt assembly.** The system prompt is byte-for-byte stable for the life of the conversation, so the provider's prefix cache never breaks. Anything new — a skill invocation, a hint — rides in a user message or a tool result, never the system prompt. With a large catalog, the model sees only three bridge tools (`tool_search`, `tool_describe`, `tool_call`).
5. **Model call.** OpenAI-format messages through the provider's transport. The model decides it needs yesterday's conversation and calls `session_search`.
6. **Dispatch.** The call goes through the single dispatch funnel — bridge unwrapping, middleware, plugin hooks, edit approval — and then the gateway's approval queue. Once approved, `session_search` runs a full-text search (with a custom tokenizer for Chinese, Japanese, and Korean) and returns yesterday's thread.
7. **Answer.** Guardrails watch for identical repeated calls; the interrupt flag is checked each iteration. The model answers.
8. **Learn.** The turn is saved, then a background thread may replay it and ask, "should any skill or memory be saved or updated?" Writes go straight to the skill and memory stores; the main conversation and its cache are untouched.

The Learning Loop

- **Background review (after turns).** The fork inherits the parent's provider, model, credentials, and cached system prompt, so it reuses the prefix cache. It runs with a restricted tool list. Reviews are controlled by settings, never run inside sub-agents, and are deferred to idle time when the model is running locally so they do not compete for the user's GPU.

- **Curator (on inactivity).** No cron daemon. When the agent is idle and the last curator run is older than the interval (7 days by default), it moves skills through lifecycle states and can optionally fork an agent to consolidate them. Rules: only curator-managed skills are touched, pinned skills are exempt, nothing is deleted — only archived, which is recoverable.
- **The write path.** The `skill_manage` tool lets the agent create, edit, and archive skills in the agentskills.io `SKILL.md` format. Review proposes, curator maintains, `skill_manage` writes.

Subsystems Worth Knowing

- **Turn phases as files.** Preflight, request assembly, tool round, overflow, compaction, stop gates, and finalizer are separate modules, so a change to one does not touch the loop.
- **Tools.** Tools register themselves at import time with a schema validated at registration. A few high-volume tools skip the registry. MCP discovery runs in a thread at startup because it once blocked the gateway for up to two minutes.
- **Approvals.** A per-session approval queue in the gateway, a YOLO bypass, and a "denial breaker" that counts refusals per session.
- **Sub-agents.** `delegate_task` children get a tool grant carved from the parent's — never wider.
- **Memory.** A provider interface with built-in, Honcho, and Hindsight implementations.
- **Providers.** About 40, each a plugin, using either Chat Completions or a Responses-style mode.

Design Tradeoffs

- **Spending calls to save calls.** Every turn may trigger a second model call; every idle week may trigger a third.
- **Cache discipline as architecture.** The rule that the system prompt never changes shapes how skills, hints, and memories are delivered.
- **Recoverability over tidiness.** Archive-never-delete keeps the skill library growing, but nothing bad is ever lost.

Security Lens

- **Self-written skills are a persistence channel.** If an attacker's text reaches a conversation (a shared chat, a webpage the agent read), the background review may distill it into a skill that loads in future sessions. Treat the skill store as executable configuration and review curator-managed and review-written skills.
- **Multi-tenant boundaries.** Session keys separate users, threads, and profiles. The article notes a known footgun where a `/model` override can miss a recovered Telegram forum-topic key — the kind of key-derivation edge worth testing for cross-session bleed.

- **YOLO bypass.** Check whether it is enabled per deployment; it removes the approval queue.
- **Unlimited by default outside the CLI.** Because the gateway falls back to an unlimited turn count when unset (see the correction above), set `max_turns` explicitly for messaging deployments.
- **Sub-agent grants never widen.** A good anti-escalation pattern.

Take-away

Learning can be a mechanism, not a slogan: a fork that proposes, a slow curator that maintains, one tool that writes, and a hard rule that nothing is ever deleted.

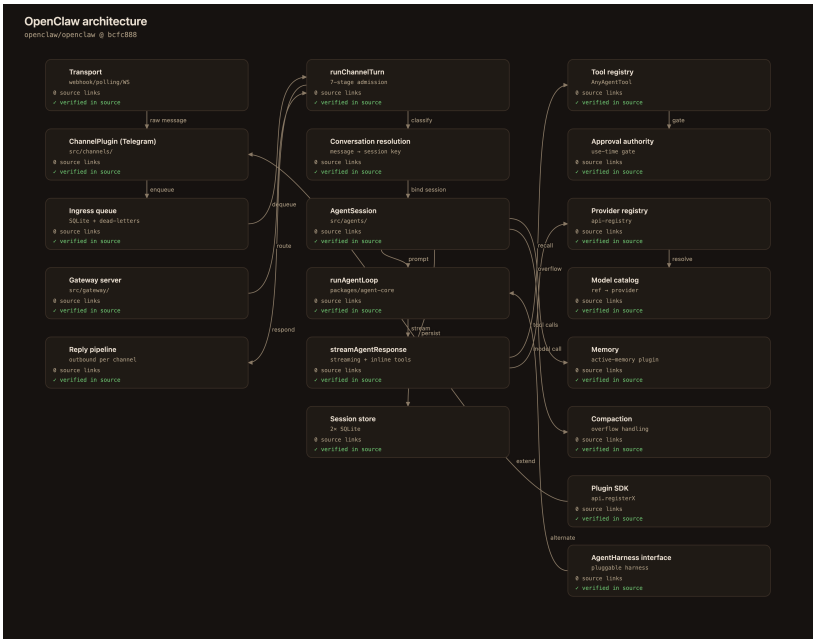
9. OpenClaw — The Channel-First Gateway

Repo: openclaw/openclaw · Pinned: v2026.9.9 → bcfc888 (Oct 8, 2026) · TypeScript (pnpm monorepo)

> **Verified.** `runChannelTurn` emits its stages in exactly the order the landing page gives — ingest, classify, preflight, resolve, assemble, dispatch, finalize; the channel layer has an ingress queue with retry policy and dead letters; the project's VISION.md gives its lineage as Warelay, Clawdbot, Moltbot, OpenClaw. No corrections.

In One Paragraph

OpenClaw treats Telegram, WhatsApp, Discord, and other messaging apps as the main way in. Every inbound message goes through a seven-stage admission pipeline before the agent loop ever sees it. A SQLite-backed ingress queue with dead letters keeps turns durable, and a Tool Search catalog loads tool definitions on demand, so most tools never enter the model's context.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

Every other harness in this series is a program you run. OpenClaw is a long-running gateway server that *hosts* agent turns. Its core abstraction is the inbound message, not the prompt. The CLI and web UI are secondary.

The channel is the product; the loop is a component. That is why the interesting question is not "how does the loop work" but "how does a WhatsApp message become an agent turn." It is also the broadest harness in the series — about 100 bundled extensions and more than 550 configuration schema files.

A Message, Step by Step

A Telegram message reading "summarize today's standup notes" arrives.

1. **Ingress.** The Telegram plugin — assembled from about 25 optional adapters (security, pairing, threading, outbound) rather than written from scratch — receives the update by long-poll or webhook. Channels run inside the gateway process.
2. **Admission.** The seven stages: *ingest* turns the raw update into normalized input (or drops it); *classify* decides the event type (a reaction, for example, is handled without a turn); *preflight* returns a verdict — dispatch, drop, handled, or observe only; *resolve* works out the turn; *assemble* binds the message to a session key; *dispatch* runs it; *finalize* closes it.
3. **Persist before execute.** Guards drop echoes of the bot's own messages and break bot-to-bot loops. The message is saved to the session store *before* the agent runs, so the turn is durable before it executes. Channels can instead enqueue into the SQLite ingress queue, which delivers at least once, in order per lane, with retries, dead letters, and a stall watchdog.
4. **Session binding.** Routing maps channel, account, and peer to an agent and session; multi-account channels resolve to separate sessions.
5. **Execution.** The four-layer loop runs against the bound session.
6. **Reply.** Output becomes channel-shaped reply payloads, sent through a durable sender and recorded as receipts. Approval prompts appear as native channel UI, such as Telegram inline buttons.

The Four-Layer Loop

1. **`runLoop`** — the raw model-and-tools loop in a package that knows nothing about channels. Tool calls run *as they arrive in the stream*, not after the response completes. New user messages can be injected mid-loop. There is no hard turn counter; stopping is delegated to `shouldStopAfterTurn` and similar hooks. A detector watches for pathological tool-call cycles.
2. **`Agent`** — owns the transcript, the steering queue, and the follow-up queue.
3. **`AgentSession`** — orchestration: compaction, extensions, retries with backoff (but context overflow goes to compaction, never retry), and post-run directives like "continue" or "handoff."
4. **Gateway execution** — 54 files of operational machinery: admission control, de-duplication, restart recovery, sub-agent runs.

An `AgentHarness` interface lets the gateway dispatch a turn to a *non-OpenClaw* harness. At this pin no such implementation ships — it is a contract awaiting implementations.

Subsystems Worth Knowing

- **Tools.** The effective tool surface merges six sources (core, shell, channel, OpenClaw, plugins, Tool Search) and then filters by sandbox, profile, provider, sender, group, and sub-agent policy.
- **Approvals.** An explicitly ordered chain where order is behavior: loop admission, owner probes, voice confirmation, trusted policies, approvals, normal hooks, final owner approval. The gateway checks a run's *delegated authority* at the moment of use, and decisions are audited with hashed receipts.
- **State.** Two SQLite databases with no external server: a shared state database and a per-agent database holding append-only transcript events.
- **Memory.** A `MEMORY.md` seed, a vector and keyword index, an active-memory plugin that surfaces relevant memories before replies, and a curation ("dreaming") subsystem.
- **Plugins.** Capability-typed: a plugin registers providers, channels, embeddings, speech, media generation, and more through `api.registerX` calls, each with a manifest. Heavy channels load lazily.
- **Skills.** Instruction packs, not tools — they steer the model toward real tools through the prompt.

Design Tradeoffs

- **Every message pays the pipeline,** even when the verdict is a trivial dispatch. In return: uniform policy, durability, and multi-tenancy.
- **Breadth means operational surface.** Hundreds of config schemas and dozens of plugins are a lot to understand and secure.
- **No runtime model failover.** A fallback chain is resolved at configuration time; a provider error mid-run does not switch providers.

Security Lens

- **The internet can talk to your agent.** Any channel that accepts messages from outsiders is an unauthenticated prompt-injection channel. The preflight verdict, pairing, and sender filters are the first line of defense; test them per channel.
- **Bot-loop and echo guards** exist because two bots in one chat can drive each other indefinitely. Verify they hold across multi-account setups.
- **Delegated authority is the right model.** Checking what a *run* is allowed to do — not just which tool is called — is the strongest approval design for a multi-user agent. Hashed receipts make decisions auditable.

- **Durability cuts both ways.** Persist-before-execute and dead-letter queues mean a malicious message that crashes a turn will be retried and stored. Review dead-letter handling and retention.
- **Lazy plugins, large surface.** About 100 bundled extensions; check which are activated at startup and which are reachable from a channel.

Take-away

When the agent lives in public channels, admission is the security boundary. A staged pipeline with explicit verdicts — before any model call — is the pattern to copy.

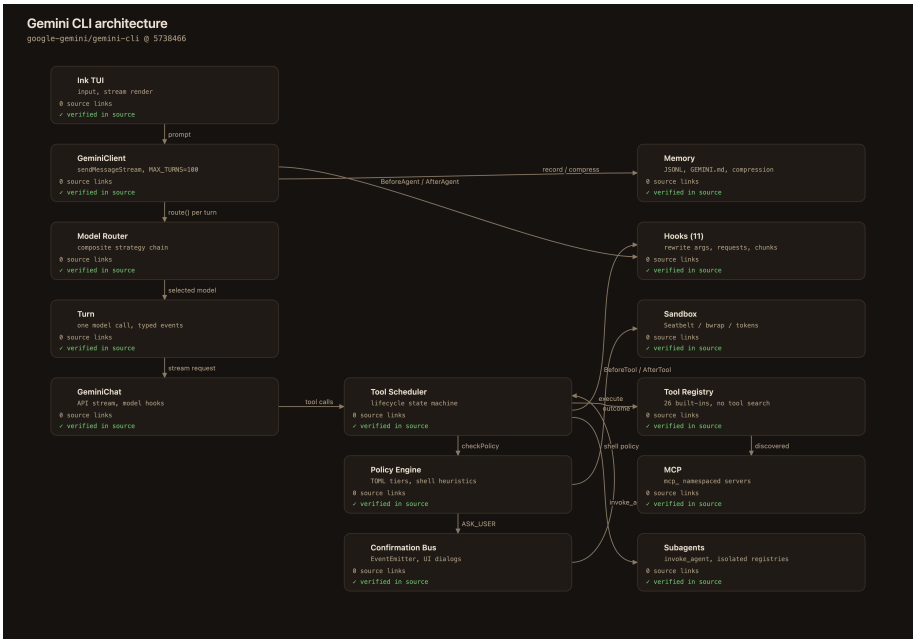
10. Gemini CLI — The Routed, Policy-Gated Terminal Agent

Repo: google-gemini/gemini-cli · Pinned: v0.63.0 → 5738466 (Oct 6, 2026) · TypeScript (npm workspaces) · Apache-2.0

> **Verified.** `MAX_TURNS = 100`; the router chain runs Fallback, Override, ApprovalMode, GemmaClassifier (opt-in), Classifier, NumericalClassifier, then Default; 11 hook events; no tool-search code; policy tiers start at `DEFAULT_POLICY_TIER = 1`.
>
> **Correction (C1).** The landing page says memory is "mined nightly." The memory service runs when the CLI starts and skips itself if the last run was under 30 minutes ago; it only considers sessions idle for at least 3 hours with at least 10 user messages. It is opportunistic and throttled, not nightly.
>
> **Correction (C8).** "All 26 built-ins ride every request." The code can register 27 tool classes, but `grep` and `ripgrep` are alternatives, so 26 is the maximum at once — and several (the to-do, plan-mode, and tracker tools) are registered only when their features are on. What is exactly true: every *active* tool is sent on every request.

In One Paragraph

Google's terminal coding agent can pick a different model *every turn* through a chain of routing strategies. Tools run through a scheduler with explicit lifecycle states, and each call is gated by a TOML policy engine with tiered priorities plus eleven hook types that can intercept almost anything. There is no tool search — every active built-in goes out with every request. Memory is JSONL session logs plus markdown files, mined periodically by a background extraction service.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

Delegation. Earlier harnesses in the series have one model, one loop, and one policy. Gemini CLI breaks all three: a router chooses the model, a scheduler runs the tools, a policy engine decides what needs asking, and hooks let extensions rewrite tool arguments, model requests, and even streamed responses.

The core loop decides little. It checks turn budgets, runs loop detection, asks the router, and streams events. Notably, the loop is split across packages: the core streams and decides; the CLI package schedules tools and feeds results back in. The price of all this machinery is predictability — roughly 138,000 lines in the core package alone, and many systems to understand before you can say what a turn will do.

A Turn, Step by Step

The user types "Summarize the changes in this repo."

1. **Input.** The Ink terminal UI submits the prompt. `BeforeAgent` hooks fire once per prompt and can stop it, block it, or add context.
2. **Turn budget.** The session turn count is checked; the request is bounded to at most 100 turns.
3. **Loop guard.** The loop detector can abort repeated patterns before the model is called.
4. **Route the model.** The strategy chain picks the model for *this* turn, and the decision is logged with its source and reasoning. Unless a sequence model is pinned, turn N+1 may use a different model than turn N.
5. **Model call.** `BeforeModel` hooks can rewrite the request; `AfterModel` hooks see each streamed chunk and can replace it. The model asks to run `git log --oneline -10`.

6. **Tool request.** The CLI hands the call to the core scheduler, which moves it through Validating, Scheduled, executing, and a terminal state.
7. **Approval.** A `BeforeTool` hook may rewrite arguments or force a question; the TOML policy engine returns allow, deny, or ask; edits to build files are escalated to ask; then a confirmation request goes over a message bus to the dialog. "Always allow" writes a new rule to an auto-saved policy file.
8. **Feed back.** The output is truncated if large and sent back as function responses, re-entering the loop. When a turn ends with no tool calls, a *separate* model call decides whether the model should keep going.

Subsystems Worth Knowing

- **Policy engine.** TOML rules matched by tool name, argument regex, annotations, approval mode, and interactivity, sorted by priority with first match winning. Tiers are structural — Default, Extension, Workspace, User, Admin — so an admin rule always beats a user rule. Shell commands are unwrapped, split into sub-commands, and each checked; redirections downgrade "allow" to "ask." Extension-supplied allow rules are dropped.
- **Confirmation bus.** Decouples policy from UI. Sub-agent buses are treated as untrusted: forced decisions are stripped before reaching the parent.
- **Safety checkers.** Run after the policy decision and fail closed to deny. **Conseca**, the notable one, has a fast Gemini model write a least-privilege policy from the conversation and then enforces it per call. It is off by default.
- **Sandbox.** macOS Seatbelt (deny by default), Linux bubblewrap, Windows restricted tokens and job objects.
- **Memory.** Sessions as JSONL. Two compaction paths: the legacy one triggers at half the token limit and summarizes the oldest 70% into a structured snapshot; an experimental context manager uses a fixed budget. Long-term memory is `GEMINI.md` files at four levels — global, extension, project, and per directory (injected just in time when a path is accessed). No vector database.
- **Hooks.** Eleven events. Shell-command hooks (60-second timeout) or in-process hooks. Project hooks are ignored in untrusted folders.
- **Sub-agents.** One tool, `invoke_agent`. Each runs with fresh, isolated registries, cannot nest, must call `complete_task` to finish, and has default limits of 30 turns and 10 minutes.
- **MCP and skills.** MCP tools are namespaced `mcp_server_tool` and look like ordinary tools. Skills load in two stages: the prompt lists names; `activate_skill` (with user confirmation for non-built-ins) injects the body.

Design Tradeoffs

- **Machinery for control.** Per-turn model choice, learned approval rules, and eleven interception points — at the cost of predictability.

- **No tool search.** Every active tool rides every request. Simple and transparent, but context-expensive as MCP servers pile up.
- **Markdown memory.** Easy to read and edit; no semantic recall. Batch extraction compensates.
- **YOLO is command-line only.** It cannot be set in a settings file — deliberate friction against a permanent foot-gun.

Security Lens

- **Hooks can replace model output mid-stream.** `AfterModel` hooks can rewrite what the model "said." Treat hook configuration as privileged; it is ignored in untrusted folders for good reason.
- **Tiered policy is a strong design.** Admin rules beat user rules regardless of numeric priority, and extensions cannot add allow rules. Assess by listing rules per tier.
- **Shell parsing is the soft spot.** The engine splits compound commands and treats redirections conservatively. Command-parsing differentials (quoting, substitutions, interpreter flags) are always worth testing against any engine that reasons about shell strings.
- **Model routing changes the safety profile per turn.** A classifier deciding which model answers means safety behavior can vary turn to turn. Routing is logged — use the logs.
- **Conseca is off by default** and fail-open when disabled; do not count it in a deployment's controls unless it is turned on.
- **MCP in untrusted folders.** Stdio MCP servers are refused there, and their environment is sanitized.

Take-away

Tier your policies by who wrote them. Making "who authored this rule" part of the precedence order — admin over user over workspace over extension — removes a whole class of policy-override bugs.

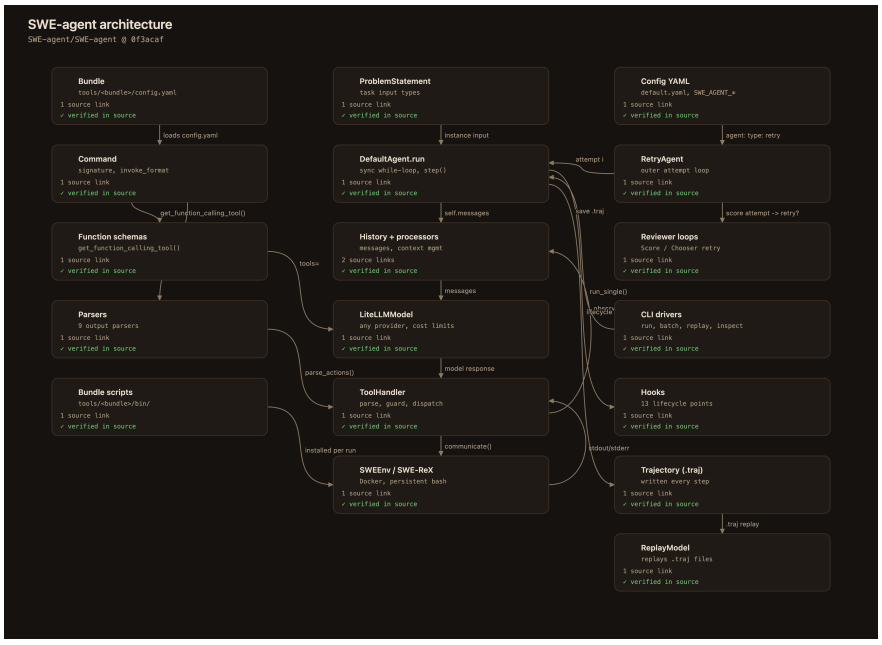
11. SWE-agent — The Research Original

Repo: SWE-agent/SWE-agent (moved from SWE-bench/SWE-agent) · Pinned: v1.1.0 → 0f3acaf (May 22, 2025) · Python 3.11+ · MIT

> **Verified.** The loop is literally `while not step_output.done`, saving the trajectory after every step; the default config loads the `registry`, `edit_anthropic`, and `review_on_submit_m` bundles (no windowed file viewer); permissions are a command blocklist; the web API ships `app.secret_key = "super secret key"`; the editor tool's docstring carries a comment saying it was taken from OpenHands.
>
> **Correction (C4).** The SWE-agent article calls it "the ancestor OpenHands descends from." That is not supported. OpenDevin (now OpenHands) and SWE-agent were separate projects that both appeared in spring 2024, and the one code-level link found runs the other way: SWE-agent borrowed OpenHands' editor docstring.

In One Paragraph

SWE-agent is a benchmark harness that grew an agent. The model never sees a raw shell; it sees a *designed* agent-computer interface (ACI) defined as data — YAML command schemas plus scripts that run inside a container. Around a deliberately simple synchronous loop sits research instrumentation: the trajectory rewritten after every step, a replay model, and reviewer loops that retry whole attempts.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

Read every design decision as research tooling. The trajectory file is rewritten each step so a crashed run is still data. A replay "model" re-executes saved runs so results can be audited. Trajectories can be converted into few-shot examples. A reviewer scores whole attempts and retries them — inference-time scaling as a config switch. When a run hits its cost limit, it submits whatever patch exists instead of dying empty-handed.

The headline idea is the ACI. Commands are declared in YAML (signature, docstring, typed arguments), implemented as scripts copied into the container, exposed to the model either as function-calling schemas or as documentation in the prompt, and dispatched as shell strings into a persistent container shell. The action space is literally edited as text files.

A Step, End to End

1. **History.** The agent's history is filtered and passed through configured history processors.
2. **Model call.** LiteLLM, non-streaming, with function schemas; temperature 0 by default. Token count is checked against the context window *before* spending a call. Cost is tracked against a per-instance limit ($3.00 by default), a total limit, and a call limit.
3. **Parse.** The function-calling parser requires exactly one known tool call with its required arguments and renders it into a shell string.
4. **Guard.** Blocklisted commands are refused; a literal `exit` ends the run; multi-line commands are wrapped in a heredoc.
5. **Execute.** The string runs in the container's persistent bash session with a 30-second timeout. Special tokens in the output (retry with output, retry without, forfeit) steer the loop and are stripped before the model sees them.
6. **Submission check.** If the output contains the submission marker, the patch at `/root/model.patch` is read and the run ends as "submitted."
7. **Feedback.** The observation goes into the next prompt template. Output over 100,000 characters is truncated with a note saying how much was cut. State commands report the working directory, open file, and diff.
8. **Bookkeeping.** The step is appended to the trajectory and the `.traj` file is rewritten.

Format errors, blocked commands, and bash syntax errors re-ask the model up to three times without polluting the permanent history; after that, the run submits what it has. Fatal errors — cost, context, timeouts — go straight to "autosubmission," which diffs the working tree into a patch.

Subsystems Worth Knowing

- **The ACI bundles.** Thirteen bundle directories. The paper-era *windowed* viewer (`open`, `goto`, `scroll`) and search commands still exist, but the default config ships the Anthropic-style `str_replace_editor` instead — a docs-versus-code gap the article flags.
- **No step cap.** Only the call limit (default unlimited), cost limits, and a 30-minute wall clock bound the loop. The SWE-bench heavy config sets 75 calls.

- **The retry agent.** An outer loop runs whole attempts, hard-resetting the environment between them. A score loop retries while a model's score is below a threshold; a chooser loop picks the best attempt. Despite its name, the reviewer never sees individual actions.
- **Special models.** `human` (you are the model), `replay` (re-run a trajectory), and a test model that submits instantly.
- **Hooks.** Thirteen agent hook points plus environment and run hooks. Pydantic forbids unknown config keys, so typos fail loudly.

Design Tradeoffs

- **Flexibility through strings.** Swap the whole interface by editing YAML — at the cost of a type system that is, in effect, a shell script in a container.
- **Instrumentation over product.** Replay, trajectories, and reviewers are superb for research and absent from most products.
- **Boring loop, interesting edges.** The synchronous loop is easy to reason about; the attempt-level retry and autosubmission carry the cleverness.

Security Lens

- **Researcher threat model.** The container (default image `python:3.11`) is the only sandbox, and the agent runs as root inside it. There are no approvals — only a blocklist (editors, debuggers, `nohup`, bare interpreters, and similar).
- **Blocklists are bypassable by design.** A blocklist stops interactive programs that would hang the session; it is not a security boundary. Anything not on the list runs.
- **Web UI secret.** The Flask API uses a hard-coded secret key. Do not expose the web UI on a shared network.
- **Autosubmission diffs the tree, it does not protect it.** There is no git safety net for the environment.
- **Good evaluation hygiene.** Per-step trajectories make every action reviewable after the fact — useful for incident analysis in any agent deployment.

Take-away

Design the interface the model sees. A small, purpose-built command surface — defined as data so it can be changed without code — often beats handing the model a raw shell.

12. Qwen Code — The Fork That Outgrew Its Parent

Repo: QwenLM/qwen-code · Pinned: v0.25.0 → 6788c03 (Oct 5, 2026) · TypeScript (pnpm workspaces) · Apache-2.0

> **Verified.** The README says Qwen Code was originally based on Gemini CLI v0.8.2 and stopped syncing at Qwen Code v0.1; there are 22 hook events; there are 13 provider preset files; the AUTO-mode classifier is documented as fail-closed; `MAX_TURNS = 100`; a `serve` command exists. No corrections.

In One Paragraph

Qwen Code is Alibaba's fork of Gemini CLI, independent since its v0.1. The fork point came *before* Gemini CLI added its policy engine and its model router, so Qwen built both areas from scratch: a permission manager based on rule strings with a default AUTO mode driven by a fail-closed language-model classifier; a registry of 13 provider presets speaking several protocols, with deep Qwen-specific wire handling; 22 hook types; Agent Teams; and a `qwen serve` daemon that fronts messaging channels, a desktop app, mobile, and three SDKs.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

Gemini CLI's complexity is vertical — router, scheduler, policy engine, sandbox, and Conseca stacked inside one terminal program. Qwen Code's complexity is horizontal: the loop stays thin while the codebase spreads across providers, runtimes, interfaces, and collaboration features.

The guiding idea shifted from *delegation* to *explicitness*. There is no per-turn router (you choose with `/model`), no automatic "should the model keep going?" check by default, and no policy tiers — just one decision order: deny, then ask, then allow, then default.

A Turn, Step by Step

1. **Input.** The terminal UI submits the prompt; `UserPromptSubmit` hooks fire.
2. **Budget.** Up to 100 turns per request, a session turn cap, and a session token limit keyed to the resolved model route. A user can type a budget directive such as `+500k` tokens.
3. **Loop guard.** Always-on safeties: 5 identical consecutive tool calls, 8 shell-inspection calls with no progress, and soft and hard caps of 100 and 1,000 tool calls per turn. Additional heuristics are opt-in.
4. **Model resolution.** No router — the content generator for the selected model and provider is resolved fresh.
5. **Model call.** `MessageDisplay` hooks fire as the reply streams. If the wire retries or falls back to another model, accumulated tool calls are cleared so nothing runs twice.
6. **Tool request.** The inherited scheduler moves the call through validating, scheduled, and executing.
7. **Permissions.** Three levels: the tool's own default (a non-read-only shell command asks), then the permission manager's rules, then the approval mode — where AUTO runs a filter chain and the classifier.
8. **Feedback.** Results re-enter the loop recursively with one fewer turn. When nothing is pending, a `Stop` hook chain runs before the turn ends.

Subsystems Worth Knowing

- **Permission rules as strings.** For example `Bash(git *)`, `Read(./secrets/**)`, `WebFetch(domain:x.com)`. Evaluation is deny, ask, allow, default; session rules before saved rules; restrictive rules follow real paths so a symlink cannot widen an allow. A whole-tool deny removes the tool from the registry.
- **Five approval modes,** with **AUTO as the default** (untrusted folders force manual approval). Unlike Gemini CLI, YOLO can be set permanently in `settings.json`.
- **The AUTO classifier.** Runs only when rules did not decide. First a fast path for edits inside the workspace, then an allowlist of read-only built-ins (MCP tools excluded), then a deterministic destructive-command guard, and only then a two-stage model classifier. Any classifier error means *block*. Protected surfaces — `.git`, `package.json`, CI workflows, Qwen's own config and memory — never take the fast path. Three consecutive blocks, two "classifier unavailable" results, or twenty total blocks fall back to manual approval.
- **Tools.** About 54 built-in tool classes versus the parent's 26, plus a `tool_search` tool Gemini CLI never built: deferred tools stay hidden until searched for.
- **Providers.** Six auth "wires" (OpenAI, OpenAI Responses, Qwen OAuth, Gemini, Vertex AI, Anthropic), 13 presets, and per-family logic such as mapping Qwen reasoning settings and parsing inline `<think>` tags.

- **Memory.** Session JSONL; three compaction tiers modeled on Claude Code; context files `QWEN.md`, `AGENTS.md`, and a git-ignored `QWEN.local.md`; and a memory manager with project, user, and team layers. Extraction runs after every user query.
- **Hooks.** Twenty-two event types in four kinds — command, HTTP (with SSRF protection), function, and prompt. Skills can register session-scoped hooks.
- **Teams and planning.** A first-class plan mode, Agent Teams with a mailbox and task board, resumable workflows and background agents, and a native LSP client.
- **Interfaces.** `qwen serve` (an ACP daemon over HTTP, SSE, and WebSocket), eleven channel adapters that each spawn a headless `qwen --acp`, a web shell, a Tauri desktop app, a VS Code companion, an Android shell, and TypeScript, Python, and Java SDKs.

Design Tradeoffs

- **Explicit over clever.** Users always know which model answers; the cost is no automatic per-turn optimization.
- **Surface area as a feature.** Many interfaces and providers mean reach — and much more to secure and keep consistent.
- **A gutted inheritance.** The confirmation bus inherited from Gemini CLI now auto-confirms with a comment "policy engine removed." Leftover structure can mislead readers of the code.

Security Lens

- **A model decides by default.** AUTO mode means a classifier approves most calls out of the box. Its design is careful — deterministic guards first, fail-closed, protected paths, circuit breakers — but any assessment should focus on what reaches the classifier and how its transcript input can be shaped.
- **The classifier sees a filtered transcript** (assistant text and tool results stripped). That reduces injection through tool output, a sensible choice to probe rather than assume.
- **Interpreter-led allow rules are removed in AUTO.** A rule like `Bash(python *)` would allow anything; Qwen strips such rules in AUTO.
- **Persistent YOLO.** Because YOLO can live in `settings.json`, check configuration files, not just command lines.
- **Sandbox is Linux-only** and incompatible with ACP, worktrees, LSP, MCP, and extensions. macOS and Windows sandboxing from the parent were removed, not replaced.
- **Channels spawn headless agents.** Eleven adapters (Telegram, WeChat, DingTalk, Feishu, GitHub, GitLab, email, and more) put outside messages in front of an agent; review the capability handshake between channel bridge and child process.
- **Unattended retry mode** can retry capacity errors indefinitely; watch it in automated environments.

Take-away

If a model is going to approve actions, put it last. Qwen's chain — rules, then fast paths, then a deterministic destructive-command guard, then the classifier, failing closed with a circuit breaker back to humans — is a good template for LLM-assisted approval.

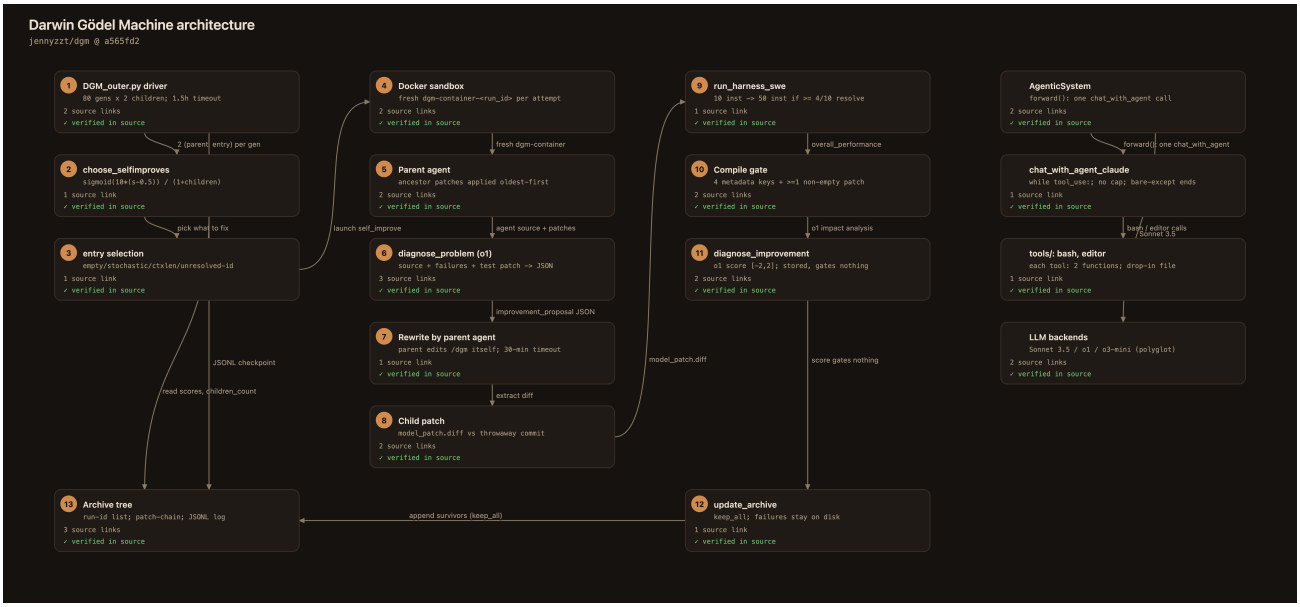
13. Darwin Gödel Machine — The Agent That Rewrites Itself

Repo: jennyzzt/dgm · Pinned: main → a565fd2 (Aug 13, 2025) · Python, flat repo of 39 files · Apache-2.0 · Paper: arXiv 2505.22954

> **Verified.** The argparse choices list is missing a comma (`'score_child_prop' 'best'`), which fuses two options; the diagnosis prompt template includes `{test_patch}` , the hidden SWE-bench test patch; the bash tool creates a new shell session on every call while its description claims state persists; the wired models are Claude 3.5 Sonnet via Bedrock, o3-mini, and o1 (2024-12-17); arXiv lists v3 on March 12, 2026, after the pinned code.
>
> **Correction (C7).** The landing page calls it "Sakana AI's." The paper lists the University of British Columbia, the Vector Institute, Sakana AI, and a Canada CIFAR AI Chair. It is a joint academic–industry project.

In One Paragraph

The Darwin Gödel Machine is an empirical attempt at recursive self-improvement. A coding agent rewrites its own source code, each new version is tested on SWE-bench, and every version that works is kept in an *archive* instead of only keeping the best. Versions that do not win right away can still become parents of later breakthroughs. The recursion is only one level deep, and the limits are real: the model that diagnoses failures sees the private test patch, and the paper's latest version is newer than the pinned code.



Architecture overview. Figure from the Agent Harness Anatomy series (Apache-2.0).

Mental Model

Two loops, not one agent. The **outer loop** is a fixed, hand-written evolutionary driver. It keeps a tree of agent versions, picks a parent, asks a strong model to diagnose why the parent failed on a task, and then asks the agent to rewrite its own code to fix that. The **inner loop** is the thing being evolved: a deliberately simple coding agent with exactly two tools, `bash` and `editor`.

The genuinely recursive part is subtle: the rewrite is performed *by the current parent agent itself*. A better coder becomes a better self-editor. That is the whole mechanism behind "improves its ability to improve," and it is real. But the driver, the diagnoser, its prompts, and the benchmark machinery never change.

The Outer Loop

Defaults: 80 generations, 2 self-improvement attempts per generation, 2 parallel workers, each attempt with a 1.5-hour timeout. Each generation:

1. **Choose parents.** Each archive member is weighted by a sigmoid of its score times one over one-plus its number of children. Good agents are preferred, but heavily used ones are discounted — an explicit bonus for exploring neglected branches. Sampling is with replacement.
2. **Choose what to fix.** A quarter of the time each: fix empty patches (if at least 10% of tasks produced none), fix randomness, fix context-length errors (if seen twice), or otherwise pick a random unresolved task.
3. **Run children in parallel.** A failed or timed-out child is logged; a generation never fails because of one child.
4. **Compile filter.** Keep only children that produced the expected non-empty patches and complete results.
5. **Update the archive.** By default *every* compiled child joins, even if worse than its parent. That is the "stepping stones, not hill-climbing" claim, in code.
6. **Checkpoint.** One JSON line per generation. That file is the entire persistent state of the evolution.

A child's full code is never stored — only its diff against its parent. Any node is rebuilt by walking the parent chain back to the initial agent and applying patches in order.

One Child, End to End

1. A fresh Docker container is started.
2. The parent is materialized by applying all ancestor patches.
3. A throwaway commit isolates this attempt's changes.
4. **Diagnose.** The o1 diagnoser receives the full source of the coding agent, the failure logs — and the hidden test patch — and returns a structured proposal.

5. **Rewrite.** A fresh copy of the *parent* agent edits the live codebase inside the container, with a 30-minute limit.
6. **Validate.** Stage 1 runs 10 tasks; only if at least 4 pass does stage 2 run 50 more, and stage 2's score replaces stage 1's. The 140-task set is never touched during evolution. The seed agent scores 0.2.

The Inner Agent

One `forward()` call wraps a native tool-calling loop with no step cap. It ends when the model stops calling tools, a timeout fires, or an exception is caught by a bare `except` that returns whatever history exists. The output is just a patch file. Two tools: `bash` (120-second timeout, a new shell each call despite the description) and `editor` (view, create, or overwrite a whole file — no partial edits). New tools are added by dropping a file in `tools/`, and the contract is explained to the model, so a child could in principle give itself new tools; that is not demonstrated at this pin.

Honest Limits

- **One level deep.** Nothing ever improves the diagnoser, the sampling method, or the evaluation.
- **Information leak.** The private test patch is pasted into the diagnosis prompt, so "general" improvements are proposed with knowledge of exactly what the hidden tests check. The answer patch is loaded too but silently dropped because the template has no slot for it.
- **Fitness is benchmark accuracy.** Nothing measures self-modification ability directly; a post-hoc score from −2 to 2 is stored and gates nothing.
- **Rough edges.** The argparse bug makes the `best` sampling strategy unreachable from the command line; several functions and thresholds are dead code; there is no token or dollar accounting.
- **Paper versus pin.** arXiv v3 (March 12, 2026) postdates the code (August 13, 2025). Treat the code as the record.
- **Reported results** (from the paper's abstract): SWE-bench from 20.0% to 50.0%, Polyglot from 14.2% to 30.7%.

Security Lens

- **Self-modifying code with the operator's keys.** API keys (Anthropic, OpenAI, AWS) are passed as environment variables into every container where an agent with `bash` runs. A self-editing agent can read them.
- **Unhardened containers.** No network restrictions, no memory or CPU limits, no dropped capabilities. The "no internet" statement in the bash tool is prompt text, not a firewall rule.

- **No human in the loop.** No approvals, no kill switch beyond timeouts. The README's safety note — that it executes untrusted, model-generated code — is the whole stated posture.
- **The real constraint is emergent.** Nothing forbids the agent from editing the outer driver inside the container; such edits are simply dropped at evaluation time because only the agent's own subtree is copied into task containers. That is a containment-by-accident pattern worth noticing.
- **Excellent lineage.** Every node keeps its diff, metadata, and the full self-modification transcript, so any version can be reconstructed and audited.

Take-away

Keep an archive, not a champion. Admitting weaker variants and rewarding under-explored branches is a simple, concrete way to avoid the local optimum that pure hill-climbing gets stuck in.

14. The Thirteen, Side by Side

The landing page links a ten-row comparison matrix. This chapter condenses it into the questions that matter most when choosing, building, or assessing a harness.

Who Stops the Loop?

Harness	Hard cap on the core loop	Other bounds
pi	None	`finishTurn` hook, Ctrl+C
aider	None on the REPL	One model call per turn; 3 reflections
Cline	None in VS Code	Abort at 5 identical calls; stop after 6 failed turns
Goose	1,000 turns	Repetition guard registered but inert
Codex CLI	None	Retry budget, compaction once per step, user interrupt
OpenHands	500 iterations per run	Cost budget; stuck detector
OpenCode	None (`maxSteps` = infinity)	Configured steps inject a wrap-up prompt
Hermes	500 in the CLI default; unlimited otherwise	Identical-call guardrails
OpenClaw	None in the core loop	Stop hooks; gateway retry budgets
Gemini CLI	100 turns per request	Session cap; loop detector
SWE-agent	None	Cost, call, and time limits
Qwen Code	100 turns per request	Session and token caps; 5-identical and 1,000-call safeties
DGM (inner)	None	Container timeouts

Ask, Contain, or Undo?

Harness	Ask (approvals)	Contain (sandbox)	Undo
pi	Only via extensions	No	No
aider	New files, outside files, every shell command	No	Git commit per turn + `/undo`
Cline	Per-tool policy + UI with diff previews	No	Stash snapshot per turn + `/undo`
Goose	Modes, permission levels, five inspectors, LLM judge	No	No

Harness	Ask (approvals)	Contain (sandbox)	Undo
Codex CLI	Per-action; hooks, Guardian, user	Four layers, OS-level	No
OpenHands	Risk analyzer + confirmation policy	Deployment (Docker)	Per-file `undo_edit` only
OpenCode	Last-match rules, default ask	No	No
Hermes	Per-session queue; YOLO bypass	No	No (skills archived, never deleted)
OpenClaw	Ordered chain; delegated authority	Policy filters	No
Gemini CLI	Tiered TOML policy; checkers	Per-OS sandbox	No
SWE-agent	None (blocklist only)	Docker, as root	No
Qwen Code	Rule strings + AUTO classifier	Linux only	No
DGM	None	Docker, unhardened	Archive of versions

How Much of the Tool Catalog Does the Model See?

Pattern	Harnesses
Everything active, every request	Gemini CLI, aider (no tools at all), SWE-agent
Rebuilt per request to fit the model	OpenCode, Cline (editor/apply_patch swap)
Hidden until searched for	Codex (`tool_search`), pi (`deferred`), Hermes (bridge), OpenClaw (Tool Search), Qwen Code (`tool_search`)
Everything is an MCP server	Goose

Where MCP Plugs In

Approach	Harnesses
Every tool, including built-ins, is MCP	Goose
MCP tools minted as native tools	Cline, Gemini CLI
MCP tools wrapped with a visible adapter	pi, Codex
Configured directly, in skills, or in plugins	OpenHands, OpenCode, Qwen Code, Hermes, OpenClaw
No MCP	aider, SWE-agent, DGM

A Second Model in the Safety Path

Harness	Judge	On failure
Goose	SmartApprove read-only judge	Fails closed (everything prompts)
Codex CLI	Guardian auto-reviewer	No opinion falls through to the user
OpenHands	LLM security analyzer; critic; prompt hooks	Hooks fail open
Gemini CLI	Conseca least-privilege policy writer	Off by default
Qwen Code	AUTO-mode classifier	Fails closed; circuit breaker to manual

Patterns Worth Stealing

1. **pi:** hook pipeline at every seam, with tool exposure levels.
2. **aider:** PageRank repo map under a token budget.
3. **Cline:** loop as a library; hosts as configuration.
4. **Goose:** ordered pipeline with the model call last.
5. **Codex:** no convenience file tools, so every action crosses the sandbox.
6. **OpenHands:** a maintained view the model sees, never the raw log.
7. **OpenCode:** compiler errors appended to the edit's own result.
8. **Hermes:** propose, curate, write — and never delete.
9. **OpenClaw:** staged admission verdicts before any model call.
10. **Gemini CLI:** policy precedence by who wrote the rule.
11. **SWE-agent:** a designed, data-defined command surface.
12. **Qwen Code:** deterministic guards before a fail-closed classifier.
13. **DGM:** an archive of variants instead of a single champion.

15. Found a Mistake?

The landing page closes with an invitation: the teardowns live or die on their grounding, every claim is pinned to a release tag, a commit, and a line-level permalink, and corrections are welcome as pull requests — with the reminder that "a correction with a permalink beats a correction with an opinion."

This chapter expands on why that policy matters and then applies it.

Why Claims Drift

- **Pins drift.** A claim true at one commit becomes false the next week. The series' answer is to state the pin and never claim more than it shows.
- **Code gets misread.** A dead branch looks alive; a default set in one file is overridden in another. The Hermes iteration default in this review is a good example: the number 500 is real, but it lives in the CLI's config loader, not in the agent class.
- **Summaries compress.** A landing-page blurb has room for one sentence per idea. "Mined nightly" and "two runtimes" are the kind of shortcuts that creep in when a careful article is squeezed into a paragraph.
- **Docs disagree with code.** The series repeatedly found documentation describing systems that the code no longer runs. When they disagree, code wins.

How to File a Good Correction

1. Check out the pinned commit, not the latest code.
2. Find the exact line that contradicts the claim and link it as a permalink.
3. Quote the claim and the line side by side.
4. Say whether the claim is wrong, imprecise, or merely outdated relative to a newer release.

Corrections From This Review

Each item below is written so it could be filed upstream as is.

C1 — Gemini CLI memory is not mined nightly

- **Claim (landing page):** memory is JSONL plus markdown, "mined nightly by a background extraction service."
- **Code (pin 5738466):** `packages/core/src/services/memoryService.ts` defines a 30-minute minimum interval between runs (`MIN_EXTRACTION_INTERVAL_MS`), a 3-hour idle requirement (`MIN_IDLE_MS`), and a 10-user-message minimum (`MIN_USER_MESSAGES`). The run is checked when the CLI starts and skipped if the last run was too recent.

- **Suggested wording:** "mined in the background — at CLI start, at most every 30 minutes — by an extraction service."

C2 — Codex has 12 rollout item types, not 14

- **Claim (comparison matrix):** "14 `RolloutItem` variants."
- **Code (pin a956835):** `codex-rs/history/src/lib.rs` line 201 defines 12 variants, including the struct variant `InterAgentCommunicationMetadata` and `RealtimeItem`. The Codex article body already says 12.
- **Suggested fix:** change the matrix cell to 12.

C3 — Hermes' 500-iteration default is CLI-only

- **Claim (Hermes article):** "`max_iterations` defaults to 500 and is shared with subagents."
- **Code (pin f97608f):** `run_agent.py` defaults `max_iterations` to `sys.maxsize` with the comment "unlimited tool-calling iterations by default"; `hermes_cli/cli_config_load.py` sets `agent.max_turns: 500`; `hermes_cli/config.py`'s `resolve_turn_limit` returns unlimited when no limit is configured, which the gateway uses. Sub-agents default to 250 (`tools/delegate_tool.py`).
- **Suggested wording:** "The CLI's default config caps a turn at 500 model calls; the agent class and gateway default to unlimited unless `max_turns` is set."

C4 — SWE-agent is not OpenHands' ancestor

- **Claim (SWE-agent article):** "the ancestor OpenHands descends from."
- **Evidence:** OpenDevin (renamed OpenHands) and SWE-agent were separate projects that both appeared in spring 2024. The only code-level link found runs the other way: `tools/edit_anthropic/config.yaml` in SWE-agent says its docstrings were taken from OpenHands' `function_calling.py`.
- **Suggested wording:** "the research original — a contemporary of OpenHands, which it later borrowed from."

C5 — OpenCode has one loop over two transports

- **Claim (landing page):** "Runs its session loop on two runtimes."
- **Evidence:** the OpenCode article and `packages/opencode/src/session/llm/AGENTS.md` describe one session loop with two LLM transports (Vercel AI SDK by default; native client behind `OPENCODE_EXPERIMENTAL_NATIVE_LLM`).
- **Suggested wording:** "Runs one session loop over two interchangeable LLM transports."

C6 — OpenHands microagents are deprecated, not gone

- **Claim (landing page):** "Microagents are dead."
- **Code (pin 54daf05):** `openhands-sdk/openhands/sdk/skills/skill.py` still loads `~/.openhands/microagents/` and project microagent directories "(legacy)" and converts them to Skills.

- **Suggested wording:** "Microagents survive only as a legacy input; AgentSkills-format skills replaced them."

C7 — DGM is a joint project

- **Claim (landing page):** "Sakana AI's empirical shot at recursive self-improvement."
- **Evidence:** arXiv 2505.22954 lists the University of British Columbia, the Vector Institute, Sakana AI, and a Canada CIFAR AI Chair.
- **Suggested wording:** "UBC and Sakana AI's empirical shot…"

C8 — Gemini CLI sends every active tool, up to 26

- **Claim (landing page):** "all 26 built-ins ride every request."
- **Code (pin 5738466):** `packages/core/src/config/config.ts` can register 27 tool classes, with `GrepTool` and `RipGrepTool` mutually exclusive, so 26 at most. The to-do, plan-mode, and tracker tools are registered only when their features are on.
- **Suggested wording:** "every active built-in (up to 26) rides every request."

Appendix A — Verification Evidence

Method. The series repository was cloned at commit `6079e18` (October 11, 2026). Each of the thirteen upstream repositories was fetched at the exact pinned commit — all thirteen resolved — and the claims were checked by reading and searching the code at that commit. Star counts and repository descriptions were read from GitHub on October 12, 2026. Paper details were read from arXiv.

Harness	Pinned commit	What was checked	Result
pi	cd32f77	Tool list in `core/tools/index.ts`; `DEFAULT_TOOL_NAMES`; 42 `*.models.ts` provider catalogs; search for iteration-cap names in loop and session packages; `diffSystemPromptSections`; `--approve` flag	All confirmed; 0 cap symbols
aider	253f036	Entries in `model-settings.yml`; `max_reflections`; `RETRY_TIMEOUT`; `cmd_` methods; `nx.pagerank`; bare `return` before edit-distance fallback; MCP references	313 / 3 / 60 / 42 / present / dead code / 0
Cline	f58bc11	`service` and `rpc` lines in `apps/vscode/proto`; `maxIterations` in session factory; loop-detection thresholds; checkpoint refs; checkpoint docs	22 services, 224 RPCs; undefined; 3 and 5; `refs/cline/checkpoints`; docs still say shadow repo
Goose	76da81c	`DEFAULT_MAX_TURNS`; `state_machine::enabled()`; `agent.rs` length; judge label; `GooseMode`; `ExtensionConfig` variants; repetition inspector; hook events; AGENTS.md	1000; four env values; 6,382 lines; confirmed; four modes; four variants, no SSE; `None`; 12; parity rule present
Codex CLI	a956835	Workspace members; `max_turns` search; `WireApi`; chat rejection; approval precedence comment; Linux sandbox sources; `exec` client; `RolloutItem`	154; only a TUI recap constant; Responses only; `CHAT_WIRE_API_REMOVED_ERROR`; "1. Hooks"; bwrap, landlock, seccomp; in-process app-server; **12 variants**
OpenHands	54daf05	`max_iteration_per_run`; `EventType`; microagent loading; builtin tools; hook types; hook failure default; routers; sandbox code	500; 5 types; **still loaded as legacy**; switch tools present; 6; ALLOW; 3 router classes; none
OpenCode	aec0b9a	Native-LLM flag; `usePatch`; LSP per-file cap; `maxSteps`; permission `findLast`; Effect imports; catalog URL	Confirmed; 20; Infinity; last match; 227 files; models.opencode.ai with override variable
Hermes	f97608f	Curator constants; archive rule; background review; tool-search bridge; iteration defaults	7 days, 2 h idle; never delete; present; present; **class unlimited, CLI 500, sub-agents 250**
OpenClaw	bcfc888	Stage events in `run-channel-turn.ts`; ingress queue files; VISION.md	Exact order confirmed; dead-letter tests present; lineage confirmed

Harness	Pinned commit	What was checked	Result
Gemini CLI	5738466	`MAX_TURNS`; router chain; `HookEventName`; tool search; registrable tools; memory-service constants	100; seven strategies in order; 11; none; 27 classes, 26 max; **30 min, 3 h idle, 10 messages**
SWE-agent	0f3acaf	Main loop; default config bundles; blocklist; Flask secret; borrowed docstring	Confirmed
Qwen Code	6788c03	README fork note; hook events; preset files; classifier docs; `MAX_TURNS`; serve	v0.8.2 / v0.1; 22; 13; fail-closed; 100; present
DGM	a565fd2	argparse choices; diagnose template; bash session; model constants; arXiv history and affiliations	Missing comma; `{test_patch}` present; new session per call; Sonnet 3.5, o3-mini, o1; v3 March 12, 2026; UBC, Vector, Sakana, CIFAR

External facts. OpenCode had about 211,700 stars and Hermes about 250,300 on October 12, 2026, consistent with the articles' "over 210,000" and "over 250,000." The `OpenHands/OpenHands` README describes Agent Canvas and points to `software-agent-sdk` for the agent.

Appendix B — Glossary

Term	Meaning
A2A	Agent-to-Agent, a protocol for agents to call other agents remotely (used by Gemini CLI's remote agents).
ACI	Agent-computer interface: the designed set of commands an agent uses instead of a raw shell (SWE-agent).
ACP	Agent Client Protocol: lets an editor or host drive an agent over a standard protocol.
AgentSkills	An open format for skills: a folder with a `SKILL.md` file and frontmatter (agentskills.io).
bubblewrap (bwrap)	A Linux tool that runs a process in new namespaces with a restricted filesystem view.
Compaction	Summarizing older conversation to fit the model's context window.
Effect	A TypeScript library for typed effects, errors, and dependency injection (OpenCode).
Fail open / fail closed	What a control does when it breaks: allow everything (open) or deny everything (closed).
Harness	The code around a model that runs the loop, tools, approvals, memory, and interfaces.
Hook	A user- or plugin-supplied callback or script that runs at a defined point in the loop.
landlock	A Linux security module that lets a process restrict its own filesystem access.
MCP	Model Context Protocol: a standard way to expose tools and resources to agents.
Prefix cache	Provider-side caching of the unchanged start of a prompt; breaking it raises cost.
Responses API	OpenAI's newer request format, used exclusively by Codex CLI.
Seatbelt	macOS's sandbox profile system, invoked through `sandbox-exec`.
seccomp	A Linux feature that filters which system calls a process may make.
Steering	Injecting a new user message into a run that is already in progress.
Tool search	Hiding most tools until the model searches for them, keeping prompts small.
YOLO mode	A mode that approves everything without asking.

Appendix C — Sources

- Agent Harness Anatomy (landing page): dpsm.github.io/agent-harness-anatomy
- Series repository (Apache-2.0): github.com/dpsm/agent-harness-anatomy
- Comparison matrix: dpsm.github.io/agent-harness-anatomy/comparison-matrix.html
- Upstream repositories at the pinned commits: earendil-works/pi, Aider-AI/aider, cline/cline, block/goose, openai/codex, OpenHands/software-agent-sdk, anomalyco/opencode, NousResearch/hermes-agent, openclaw/openclaw, google-gemini/gemini-cli, SWE-agent/SWE-agent, QwenLM/qwen-code, jennyzzt/dgm
- Darwin Gödel Machine paper: arxiv.org/abs/2505.22954 (v3, March 12, 2026)
- OpenHands paper (formerly OpenDevin): arxiv.org/abs/2407.16741
- Agent Canvas README: github.com/OpenHands/OpenHands